Escuela
Politécnica
Superior

Sistema Kubeflow para ejecución de modelos de Deep Learning



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:
Julio Corbalán Moreno
Tutores:
Marcelo Saval Calvo
Ismael Beviá Ballesteros

Diciembre 2025

Universitat d'Alacant
Universidad de Alicante

Sistema Kubeflow para ejecución de modelos de Deep Learning

Autor
Julio Corbalán Moreno

Tutores
Marcelo Saval Calvo
Tecnología Informática y Computación
Ismael Beviá Ballesteros
Tecnología Informática y Computación



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Diciembre 2025

Resumen

La inteligencia artificial es algo que está muy presente a día de hoy. El entrenamiento de modelos, creación de pipelines y el análisis de datos son algunas de las tareas que realizan tanto los investigadores como ingenieros en este campo de la informática.

La necesidad de realizar estas tareas de forma eficiente, escalable, segura y concurrente sobre los equipos que se disponen es un problema habitual. El presente Trabajo de Fin de Grado surge como respuesta a la necesidad de estos profesionales en el Departamento de Tecnología Informática y Computación de la Universidad de Alicante.

Este trabajo describe los conceptos y tecnologías existentes que están relacionadas con el sistema a implementar, nombrando algunas de las soluciones más populares, utilizando varias de ellas para lograr la solución final, junto con la experimentación que se ha realizado previa a la implementación.

Finalmente, se proporciona un manual de despliegue y configuración para el entorno Kubeflow, desde cero. Este sistema se apoyará sobre nodos que ejecutan Talos Linux, sobre un nodo de Proxmox Virtual Environment, permitiendo al departamento usar los recursos físicos de forma eficiente y colaborativa.

Abstract

Artificial inteligente is something very present nowadays. The training of models, creation of pipelines and the data analysis are some of the tasks done by researchers and engineers alike, in this field of knowledge.

The need to be able to make this tasks in an efficient, safe and concurrent form on the computers avaliable to them is a common issue. This present thesis appears as a response to the need of this professionals in the Department of Information Technology and Computing of the University of Alicante.

This thesis describes the existing concepts and technologies related to the implemented system, naming some of the most popular solutions, and utilizing some of them to reach the final solution, together with the experimentation made previous to the implementation.

Finally, a deployment manual for Kubuflow, from the ground up, is provided. This system will be deployed on nodes running Talos Linux, on top of a sigle node of Proxmox Virtual Environment, allowing the department to use the physical resources in an efficient and colaborative way.

A mi familia, especialmente mis padres, por enseñarme
todo lo que sé, y permitirme llegar hasta donde he llegado.
A mí tío Fran, por introducirme en este mundo de la informática desde pequeño.
A mi abuelo Luis, que no pudo llegar a ver su nieto acabar sus estudios.

Índice general

1 Introducción **1**
- 1.1 Motivación y contexto . . . 1
- 1.2 Objetivos . . . 2

2 Conceptos y Soluciones existentes **3**
- 2.1 Virtualización de hardware . . . 3
  - 2.1.1 *Input Output Memory Management Unit* . . . 3
- 2.2 Hipervisores . . . 3
- 2.3 Contenedores . . . 5
  - 2.3.1 Motor de contenedores . . . 5
- 2.4 Kubernetes . . . 6
  - 2.4.1 Plataformas de *Kubernetes* . . . 6
  - 2.4.2 Distribuciones de *Kubernetes* . . . 7
- 2.5 Distribuciones de Linux . . . 8
- 2.6 *Machine Learning Operations* . . . 9
  - 2.6.1 MLflow . . . 9
  - 2.6.2 Kubeflow . . . 9

3 Prototipado y pruebas previas **11**
- 3.1 Pruebas previas . . . 11
- 3.2 Instalación en el laboratorio . . . 12

4 Solucion final y Guía de despiegue **15**
- 4.1 Resumen . . . 15
- 4.2 Configuración de la BIOS . . . 16
- 4.3 Instalación de Proxmox Virtual Environment . . . 16
- 4.4 Creación y configuración de las máquinas virtuales . . . 18
  - 4.4.1 Instalación de Debian en la máquina que administrará el clúster . . . 19
  - 4.4.2 Instalación de Talos OS en el resto de máquinas virtuales . . . 28
  - 4.4.3 Instalación de TrueNAS SCALE . . . 32
- 4.5 Levantamiento del clúster . . . 42
- 4.6 Configuración del almacenamiento del clúster . . . 44
- 4.7 Arranque de *KubeFlow* . . . 47
- 4.8 Gestión de usuarios . . . 48

5 Conclusiones **49**

Bibliografía **51**

Lista de Acrónimos y Abreviaturas **55**

Índice de figuras

2.1 Diagrama de los tipos de hipervisores. 4
2.2 Diagrama de la arquitectura de Kubernetes (K8s). 6

3.1 Fotografía del ordenador utilizado para las pruebas 11
3.2 Scripts y máquinas virtuales con K3s corriendo el nodo de Proxmox, junto con proyectos personales. 12
3.3 Diagrama de utilización del registro local de Docker. 13
3.4 Problemas descargando las imágenes de Talos Linux en el laboratorio. . . 13
3.5 El sistema levantado visto desde kubectl. 14

4.1 Diagrama de la solución final. 15
4.2 Advertencia del navegador al conectar a Proxmox VE. 17
4.3 Diálogo de autentificación en Proxmox VE. 17
4.4 Advertencia de suscripción inválida. 18
4.5 Nodo de Proxmox VE vacío. 18
4.6 Menú desplegable mostrado con el click derecho 19
4.7 Pantalla que muestra los ISOs que tenemos descargados 19
4.8 Descarga del ISO de Debian desde URL 20
4.9 Progreso de la descarga del ISO de Debian 20
4.10 Creación de una nueva máquina virtual 21
4.11 Resumen de los ajustes seleccionados en la máquina virtual Debian . . . 21
4.12 Máquina virtual Debian sin arrancar 22
4.13 Menú inicial del instalador de Debian 22
4.14 Resumem del particionado de discos en el instalador Debian 23
4.15 Selección de programas en el instalador de Debian 23
4.16 Eliminación del ISO de instalación en la máquina virtual Debian 24
4.17 Primer logueo en la máquina virtual Debian 24
4.18 Primera conexión ssh a la máquina virtual Debian 25
4.19 Instalación y configuración de sudo en la máquina virtual Debian 25
4.20 Modificación realizada con visudo 26
4.21 Instalación de curl 26
4.22 Instalación de talosctl 27
4.23 Instalación de kustomice, kubectl y helm 27
4.24 Instalación de git 27
4.25 Factoría de imágenes de Talos Linux 28
4.26 Advertencia del navegador. 29
4.27 Diálogo de autentificación. 29
4.28 Opciones escogidas en la factoría de imágenes de Talos Linux 30

4.29 Descarga del ISO de Talos Linux desde URL 30
4.30 Opciones elegidas para los nodos maestros 31
4.31 Opciones elegidas para los nodos trabajadores 31
4.32 Nodo de Talos Linux arrancado, a la espera de ser configurado 32
4.33 Página de descarga del ISO de TrueNAS 32
4.34 Descarga del ISO de TrueNAS desde URL 33
4.35 Opciones elegidas para la máquina virtual TrueNAS 33
4.36 Comandos utilizados para indicar un número de serie al disco 34
4.37 Modificación realizada en 301.conf 34
4.38 Vista del disco y su número de serie desde Proxmox VE 34
4.39 Menú inicial del instalador de TrueNAS 35
4.40 Menú para escoger los discos a utilizar durante la instalación de TrueNAS 35
4.41 Configuración del usuario administrador durante la instalación de TrueNAS 36
4.42 Configuración de la contraseña del administrador durante la instalación de TrueNAS 36
4.43 Final de la instalación de TrueNAS 37
4.44 Desconexión del ISO en la máquina virtual TrueNAS 37
4.45 Consola de la máquina virtual TrueNAS tras el arranque 38
4.46 Advertencia del navegador al conectar a TrueNAS SCALE. 38
4.47 Diálogo de autentificación en TrueNAS SCALE. 39
4.48 Dashboard de TrueNAS SCALE. 39
4.49 Asistente de creación de *pools* en TrueNAS SCALE. 40
4.50 Creación de la *pool* en TrueNAS SCALE. 40
4.51 Pestaña *Shares* en TrueNAS SCALE. 41
4.52 Creación de una *share* NFS en TrueNAS SCALE. 41
4.53 Mapeado de usuarios en una *share* NFS en TrueNAS SCALE. 42
4.54 Diálogo automático para activar el servicio NFS en TrueNAS SCALE. . . 42
4.55 Nodo esperando a que se realice el bootstrap del servidor 43
4.56 Arranque del clúster, obtención de *kubeconfig*, configuración de *talosctl* . 43
4.57 Modificación realizada en _out/talosconfig 44
4.58 Resultado de ejecutar *talosctl dashboard* 44
4.59 Instalación y arranque del *driver* NFS CSI 45
4.60 Comprobación del funcionamiento de la *StorageClass* 47
4.61 Contenido de *Despliegue de KubeFlow* 47
4.62 Pantalla de bienvenida de Kubeflow 48

Índice de Códigos

4.1 Contenido de *nfs-csi.yaml* 45
4.2 Contenido de *foo-pv.yaml* 46
4.3 Contenido de *foo-pvc.yaml* 46

1 Introducción

En los últimos años, la evolución de los sistemas de Inteligencia Artificial ha revolucionado las áreas de trabajo, impulsada por el desarrollo de arquitecturas cada vez más complejas y por el aumento continuo de la necesidad de capacidad computacional. Este progreso ha hecho imprescindible la construcción de entornos de trabajo optimos, capaces de diseñar y entrenar modelos, la gestión de experimentos y la monitorización de los procesos de aprendizaje, o la realización de *pipelines*.

Con el auge de los modelos de Inteligencia Artificial, con su crecimiento exponencial en coste y tamaño, su desarrollo se vuelve más complejo, y la colaboración entre usuarios un requisito. Debido a esto, el uso de *software* que permita el uso de los recursos físicos a gran escala y de forma colaborativa es una necesidad.

Sin embargo, la configuración de un entorno que haga uso de todos los recursos de forma eficiente, y que permita la colaboración entre usuarios no es trivial, ya que se deben tener conocimientos en varias areas y tecnologías informáticas, y disponer de los recursos para realizar desarrollos y mantenimientos. Debido a esto, han surgido varios sistemas como respuesta, diseñados para simplificar los flujos de trabajo de los equipos en tareas relacionadas con el *Machine Learning*.

El objetivo de este Trabajo de Fin de Grado es la implementación de uno de estos sistemas que permita hacer un uso eficiente de los recursos computacionales en el Departamento de Tecnología Informática y Computación de la Universidad de Alicante.

1.1 Motivación y contexto

En este momento, el departamento de Tecnología Informática y Computación dispone de un equipo de reciente adquisición, que cuenta con un Intel® Core™ i9-13900KF, 64 Gigabytes de y dos tarjetas gráficas GeForce RTX 4090, sobre el que se desea entrenar modelos de Inteligencia Artificial altamente exigentes y de elevado coste computacional, así como realizar tareas relacionadas, de forma colaborativa y garantizando una utilización eficiente de los recursos disponibles.

Durante una clase de Ingeniería de Mantenimiento de Computadores y Redes, mi tutor me propuso estudiar e implantar un sistema que permitiese trabajar con este equipo del departamento de Tecnología Informática y Computación como Trabajo de Fin de Grado. Resultó ser una oportunidad muy interesante de ampliar y ejercer mis conocimientos de administración de servidores en un entorno más allá del *hobby*.

Con la experiencia que he adquirido en proyectos personales, y los conocimientos claves

aprendidos en la carrera, destacando materias como Administración De Sistemas Operativos Y De Redes De Computadores o Sistemas Operativos. Se realizaron pruebas completas sobre una máquina personal y, debido a exigencias de tiempo, se ha realizado una implementación parcial en el equipo del departamento. Por este motivo, se ha reorientado el proyecto hacia la elaboración de una guía de instalación del sistema, para poder completar su implementación en un futuro.

1.2 Objetivos

El objetivo principal de este proyecto es implementar de un sistema para el entrenamiento de modelos de Inteligencia Artificial de forma colaborativa, multiusuario, remota y escalable.

Para alcanzar este objetivo general, se plantean los siguientes subobjetivos específicos:

- Ser robusto y escalable, llevándolo al terreno de la alta disponibilidad.
- Gestionar los usuarios de dicho sistema, con posibilidad de integración con LDAP, SAML 2.0, o el sistema que utilize la Universidad de Alicante o en el departamento de Tecnología Informática y Computación.
- Encontrarse documentado, tanto la solución propuesta como las tecnologías que utiliza, de cara a la futura implementación y mantenimiento del sistema.

Por último, a nivel personal, los objetivos son demostrar, reforzar y ampliar mis conocimientos en Kubernetes y la virtualización de sistemas.

2 Conceptos y Soluciones existentes

En este apartado se explicarán algunos de los conceptos y soluciones que existen a día de hoy, que se han investigado, y sobre los que me apoyaré para la realización de este trabajo.

2.1 Virtualización de hardware

La virtualización informática (Rodríguez-Haro y cols. (2012)) también llamada virtualización de *hardware* es el concepto y la tecnología que permite a los computadores imitar las características físicas de otro computador o de un conjunto de computadores, lo que da lugar a uno o varios sistemas informáticos virtuales, con características, arquitecturas y *software* que pueden ser distintos entre sí, en un solo computador.

2.1.1 *Input Output Memory Management Unit*

La IOMMU (*IOMMU Architectural Specification* (2011)) es un componente físico del procesador que extiende la arquitectura del sistema AMD64, y permite el acceso a dispositivos físicos de manera segura en entornos virtualizados de forma directa. A este acceso directo se le suele llamar *Device Passtrough*.

2.2 Hipervisores

Un hipervisor (*What is a hypervisor?* (2023)) es el nombre que recibe una capa de *software* que implementa la virtualización descrita anteriormente. El computador donde se ejecuta el hipervisor es comúnmente denominado *host* o anfitrión, y las máquinas virtuales que se ejecutan en él como *guests* o huéspedes.

El hipervisor presenta al sistema operativo huésped con *hardware* virtualizado (con posibilidad de conexión directa con algunos dispositivos del anfitrión) sobre el que se ejecuta sus instrucciones, que luego son pasadas al anfitrión y ejecutadas casi sin pérdidas de rendimiento.

Se suelen categorizar en dos tipos, hipervisores de tipo 1 y de tipo 2, como muestra la figura 2.1. La diferencia radica en si existe un sistema operativo completo entre el *hardware* y las máquinas virtuales. Si dicho sistema operativo existe de forma independiente del hipervisor, éste se puede utilizar como un computador o servidor normal, por lo que el sistema es más versátil, pero se sacrifica recursos que se podrían dedicar a la virtualización.

Algunos de los hipervisores que existen y mas populares son:

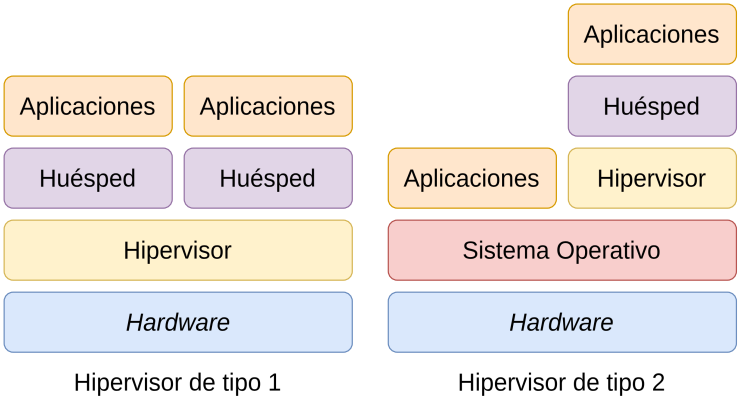


Figura 2.1: Diagrama de los tipos de hipervisores.

- **VMware ESXi:** VMware ESXi (*VMware ESXi* (2024)) (antiguamente ESX) es un hipervisor de tipo 1 y categoría empresarial desarrollado por VMware, subsidiaria de Broadcom desde finales del año 2023. ESXi se compone de un kernel propio, de código cerrado y licencia propietaria, llamado vmkernel.

  En febrero de 2024, VMWare anunció que retiraría la versión gratuita de VMware vSphere, al mismo tiempo que incrementó drásticamente el precio de las licencias anuales.

- **Microsoft Hyper-V:** Hyper-V (*Hyper-V virtualization in Windows Server and Windows* (2025)) es un hipervisor de tipo 1 desarrollado por Microsoft. Permite la ejecución de máquinas virtuales sobre sistemas AMD64 que ejecutan ejecutan Windows. Es de código cerrado y licencia propietaria.

  Se incluye en las ediciones Pro y Enterprise de Windows como una opción adicional que se puede activar. También se distribuye una versión gratuita llamada Hyper-V Server, que se conforma de una versión limitada de Windows Server y el componente Hyper-V.

  El soporte para la versión gratuita acabó el 9 de enero, 2024, (*Hyper-V Server 2019* (2025)) aunque el soporte empresarial extendido acabará el 9 de enero, 2029. El soporte como componente de Windows Server está previsto actualmente hasta el fin de soporte de Windows Server 2022, el 14 de Octubre, 2031.

- **Proxmox Virtual Environment:** *Proxmox Virtual Environment* (Proxmox VE) (*Proxmox Virtual Environment* (2025)), *PVE* o comúnmente llamado *Proxmox VE*, es una plataforma de virtualización de tipo 1, desarrollada por Proxmox Server Solutions GmbH.

  Está basada en un *kernel* Debian LTS modificado. Soporta la virtualización a nivel de *hardware* con KVM y a nivel de contenedor con LXC, dos soluciones de código abierto. También incluye una interfaz de mantenimiento web.

  Proxmox VE se entrega bajo la GNU AGPLv3, por lo que se trata de código abierto, y software libre, que se puede utilizar sin licencia. Para los clientes empresariales, dispone de varias licencias, con un repositorio privado y diferentes niveles de soporte.

- **Oracle VirtualBox:** Oracle VirtualBox (*About Oracle VirtualBox* (2025)) es una aplicación multiplataforma (por lo que se trata de un hipervisor de tipo 2) para la ejecución de máquinas virtuales, desarollada por Oracle. Se entrega bajo la GNU AGPLv3, por lo que su uso es gratuito.
- **VMware Workstation:** VMware Workstation (*VMware Fusion & Workstation* (2025)) es un hipervisor de escritorio (tipo 2), que permite la ejecución de varias máquinas virtuales al mismo tiempo. Se puede ejecutar en Windows y en Linux. Desde marzo de 2025 su uso gratuito, pero sigue siendo de código cerrado.

2.3 Contenedores

La virtualización a nivel de sistema operativo (Laadan y Nieh (2010)), también llamada virtualización basada en contenedores, o contenerización, es un método de virtualización en el que, sobre el núcleo del sistema operativo, se ejecuta una capa de virtualización que permite existan múltiples instancias aisladas de espacios de ejecución y ficheros, en lugar de solo uno. Tales instancias, las cuales son llamadas contenedores (*containers* en inglés), contenedores de *software*, jaulas (*jails* en inglés) o prisiones, pueden verse y sentirse como un servidor real desde el punto de vista de los programas. Con la virtualización basada en contenedores, se elimina gran parte de la sobrecarga asociada con tener a cada huésped ejecutando un sistema operativo completamente aislado. Aunque el concepto se originó en FreeBSD, se suele utilizar Linux para desplegar aplicaciones en contenedores.

2.3.1 Motor de contenedores

Al *software* que permite el alojamiento de distintos contenedores se le llama motor de contenedores (*container runtime* en inglés). Se encarga de la gestión de toda la vida de los contenedores, su descarga, arranque y parada, o eliminación. Estos son los motores de contenedores mas utilizados a día de hoy:

- **containerd:** containerd (*An industry-standard container runtime with an emphasis on simplicity, robustness and portability* (2025)) es un motor de contenedores considerado el estándar en la industria. Disponible como un *daemon* tanto para Linux como para Windows, es capaz de administrar el ciclo de vida del contenedor al completo. Como proyecto amparado bajo la *Cloud Native Computing Foundation* (CNCF), es de código abierto.
- **cri-o:** *cri-o* (*Lightweight Container Runtime for Kubernetes* (2023)) es un motor de contenedores ligero diseñado para su uso bajo Kubernetes, ya que implementa la Kubernetes *Container Runtime Interface* (CRI). Fue creado como alterativa a containerd por Red Hat, IBM y otras compañías.

2.4 Kubernetes

Kubernetes (K8s) (*Kubernetes Overview* (2024)) es una plataforma de orquestración de contenedores de código libre, para la automatización del despliegue, ajuste de escala y manejo de aplicaciones en contenedores que fue originalmente diseñado por Google y donado a la CNCF. Soporta diferentes entornos para la ejecución de contenedores, siempre que sean compatibles con la Kubernetes CRI.

K8s agrupa una o más computadores, virtuales o físicos, en un agrupamiento que puede ejecutar contenedores de forma organizada. Su versatilidad y utilidad para todo tipo de tareas y cargas de trabajo de todos los tamaños ha contribuido a su rápida adopción en empresas y centros de datos por igual. Al agrupamiento de estos computadores se le llama clúster.

El *software* consiste en un plano de control, formado por los nodos que se encargan de la orquestración y el control, y el resto de nodos, donde se ejecutan las aplicaciones (*Kubernetes Components* (2024)). A parte, se distribuyen herramientas para su administración como *kubeadm* y *kubectl*, que interactuan con su *Application Programming Interface* (API) pública. La figura 2.2 muestra la arquitectura descrita de forma gráfica.

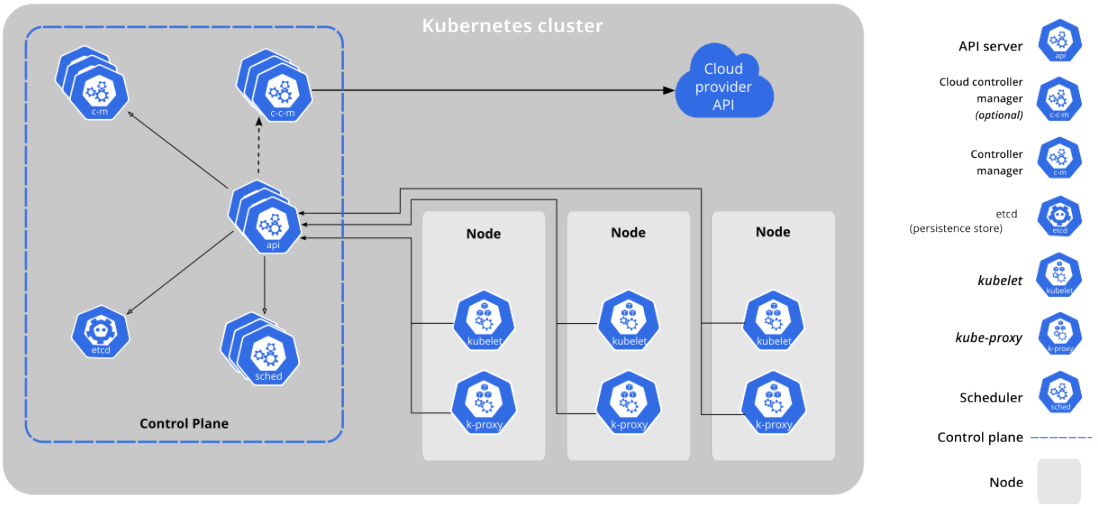


Figura 2.2: Diagrama de la arquitectura de K8s.

2.4.1 Plataformas de *Kubernetes*

Existen plataformas que simplifican la gestión de *K8s*, como la actualización, la autoescalabilidad, o la seguridad. Generalmente estan orientadas a entornos en la nube, descritas como *K8s as a Service* (*KaaS*), ya que permiten su despliegue y gestión, haciendo que el usuario no deba encargarse de la configuración manual de la infraestructura subyacente. Algunas de las plataformas más populares son:

- **Google Kubernetes Engine:** Google Kubernetes Engine (*Google Kubernetes Engi-*

ne (GKE) (2025)), es una distribución y plataforma de orquestación de contenedores diseñada para la nube y ofrecida por Google Cloud. Simplifica las fases de vida de los contenedores, el escalado y integra de forma nativa con otros servicios de Google.

- **Azure Kubernetes Service:** Azure Kubernetes Service (*Azure Kubernetes Service (AKS)* (2025)), es una distribución y servicio de K8s para desplegar y mantener aplicaciones contenedorizadas. Redice a complejidad y el coste técnico de gestionar K8s.
- **Amazon Elastic Kubernetes Service:** Amazon Elastic Kubernetes Service (*What is Amazon EKS?* (2025)), es un servicio de K8s totalmente gestionado que permite ejecutar K8s tanto en la nube de AWS (Amazon Web Services) como en centros de datos locales.
- **SUSE Rancher:** SUSE Rancher (*Rancher* (2025)) es una plataforma y un conjunto de herramientas de código abierto para gestionar múltiples clústeres de K8s en diferentes entornos desde una única interfaz desarrollada por SUSE, quien ofrece soporte empresarial. Está enfocada en facilitar la administración multi-clúster y la seguridad.
- **Red Hat OpenShift Container Platform:** Red Hat OpenShift Container Platform (*Red Hat OpenShift* (2025)), desarrollado por Red Hat, es una plataforma muy popular, diseñada para entornos híbridos de K8s, que ofrece un conjunto integral de herramientas y servicios para optimizar todo el ciclo de vida de las aplicaciones.

2.4.2 Distribuciones de *Kubernetes*

También existen distribuciones de K8s, que añaden o modifican componentes del proyecto principal de K8s, para adapatarlo a diferentes entornos o necesidades.

- **SUSE Rancher K8s Engine 2:** Rancher K8s Engine 2 (*Introduction | RKE2* (2025)) es una distribución de código abierto de K8s desarrollada por SUSE, diseñada para simplificar el despliegue y mantenimiento de clústeres de K8s.
- **Red Hat OpenShift K8s Engine:** Red Hat OpenShift K8s Engine (*Red Hat OpenShift Kubernetes Engine* (2025)) ofrece una versión reducida de las herramientas y servicios que se encuentran en la Red Hat OpenShift Container Platform, aunque manteniendo algunas ayudas a la hora de administrar clústeres de K8s.
- **MicroK8s:** MicroK8s (*The effortless Kubernetes* (2025)) es una distribución ligera de K8s desarrollada por Canonical. Es ideal para desarrollo y entornos IoT, debido a su poco tamaño y facilidad para restablecer el clúster.
- **K3s:** K3s (*K3s* (2025)) es una distribución ligera de K8s, desarrollada por Rancher Labs (parte de SUSE), ideal para dispositivos con recursos limitados.
- **k0s:** k0s (*Zero Friction Kubernetes* (2025)) es una distribución de K8s simple, diseñada para cualquier infraestructura, desde *bare-metal* y *cloud* hasta *IoT*. Es de código abierto y desarrollado por Mirantis, la empresa detrás de Docker.
- **minikube:** minikube (*minikube | Welcome!* (2025)) simplifica la creación de clústeres locales virtuales para el aprendizaje y el desarrollo local en macOS, Linux, y Windows,

utilizando máquinas virtuales o contenedores para simular los nodos. Es un proyecto de Kubernetes.

- **kind:** kind (*kind* (2025)) es una herramienta para la ejecución de clústeres de K8s locales usando contenedores de docker que actúan como nodos. Se diseño para probar K8s, pero puede ser utilizado para desarrollo local. También es un proyecto de Kubernetes.

2.5 Distribuciones de Linux

Para la ejecución de Kubernetes, necesitamos computadores (reales o virtuales) que ejecuten Linux y un motor de contenedores compatible. A continuación listo algunas de las distribuciones más comunes.

- **Ubuntu Server:** Ubuntu Server (*Ubuntu Server documentation* (2025)), mantenido por Canonical, es una de las distribuciones más utilizadas en servidores y entornos de nube gracias a su estabilidad y facilidad de uso.
- **Debian:** Debian (*The Debian GNU/Linux FAQ* (2024)) es una distribución del sistema operativo GNU/Linux, conocida por ser muy estable, segura y totalmente libre. La desarrollan y mantienen miles de voluntarios, y sirve de base para otras distros como Ubuntu.
- **SUSE Linux:** SUSE Linux Enterprise Server (SLES) (*SUSE Linux Enterprise Server* (2025)) es una distribución enfocada en el ámbito empresarial con alta disponibilidad y herramientas de gestión avanzadas.
- **Red Hat Enterprise Linux:** Red Hat Enterprise Linux (RHEL) (*Red Hat Enterprise Linux* (2025)) es una distribución comercial respaldada por Red Hat, ampliamente usada en empresas e instituciones públicas.
- **CoreOS:** CoreOS (*Fedora CoreOS Frequently Asked Questions* (2025)), ahora integrado como Fedora CoreOS, es una distribución ligera orientada al despliegue de contenedores y clústeres de K8s.
- **Flatcat Container Linux:** Flatcar Container Linux (*Flatcar Container Linux* (2025)) es una distribución Linux minimalista y orientada a contenedores, diseñada para ejecutar cargas de trabajo en K8s y otros orquestadores de contenedores. Está pensada para ser inmutable, fácilmente actualizable de forma automática y muy estable en entornos de producción en la nube o en clústeres.
- **Talos Linux:** Talos Linux (*What is Talos Linux?* (2025)) es una distribución minimalista orientada exclusivamente a K8s. El sistema es inmutable y no permite acceso mediante *shell* tradicional. EL objetivo es abstraer al usuario del sistema y que únicamente tenga que preocuparse por el mantenimiento de K8s.

2.6 *Machine Learning Operations*

Machine Learning Operations (MLOps) (Kreuzberger y cols. (2023)) es un conjunto de buenas prácticas, herramientas y cultura a la hora de trabajar, investigar y desarrollar con modelos de Inteligencia Artificial, que evolucionó del paradigma DevOps. Su objetivo es implementar y mantener los modelos de aprendizaje automático de forma fiable y eficiente. El término proviene de la fusión de *Machine Learning* y *Development Operations*.

Hoy en día, aún es común que los investigadores realicen gran parte de los flujos de trabajo realacionados con *Machine Learning* de manera manual. *MLOps* surgió como respuesta a estos desafíos del despliegue, mantenimiento e integración continua de modelos de aprendizaje automático.

Las primeras plataformas diseñadas para *MLOps* aparecieron en 2018, tras el auge en investigación relacionada. Las más populares son:

2.6.1 MLflow

MLflow (*What is MLflow?* (2024)) es una plataforma de código abierto, desarrollada por databricks, lanzada en 2018, diseñada para el manejo de flujos de trabajo relacionados con *Machine Learning*. Es habitual añadirla junto a un entorno de desarrollo e investigación ya existente, para manejar modelos y monitorizar la experimentación.

2.6.2 Kubeflow

Kubeflow (*What is Kubeflow?* (2025)) es una plataforma completa de código abierto, desarrollada iniciada por Google y ahora mantenida por la CNCF, que se ejecuta sobre K8s. Incorpora todas las herramientas que puedan ser necesarias durante flujos de trabajo con modelos de *Machine Learning* y MLOps. Es mucho más compleja que MLflow, y suele desplegarse de forma independiente.

3 Prototipado y pruebas previas

En este apartado se van a comentar algunas de las pruebas que se realizaron durante el proceso de prototipado del sistema, y la implementación parcial que se realizó en el laboratorio.

3.1 Pruebas previas

Para realizar pruebas antes de utilizar el equipo de la universidad, se usó un pequeño ordenador en casa. El equipo fué un HP EliteDesk 800 G4, el cual podemos ver en la figura 3.1, que dispone de un Intel® Core™ i5-8500T, 32 Gigabytes de RAM y 1 Terabyte de almacenamiento NVME, sobre el que ya tenía Proxmox instalado.



Figura 3.1: Fotografía del ordenador utilizado para las pruebas

Creé seis máquinas virtuales con *ubuntu-24.04-server* para realizar pruebas con Kubernetes, MLflow y Kubeflow. La primera de ellas se dejó como máquina administradora del clúster, y en las otras cinco se instalo K8s sobre ellas con K3s, generando varios *scripts* en el proceso para agilizar el despliegue durante las pruebas a la hora de montarlo sobre el ordenador encontrado en el laboratorio del departamento, como se puede ver en la figura 3.2.

Aquí se probó tanto MLflow como Kubeflow, y se decidió que este segundo era el que mejor

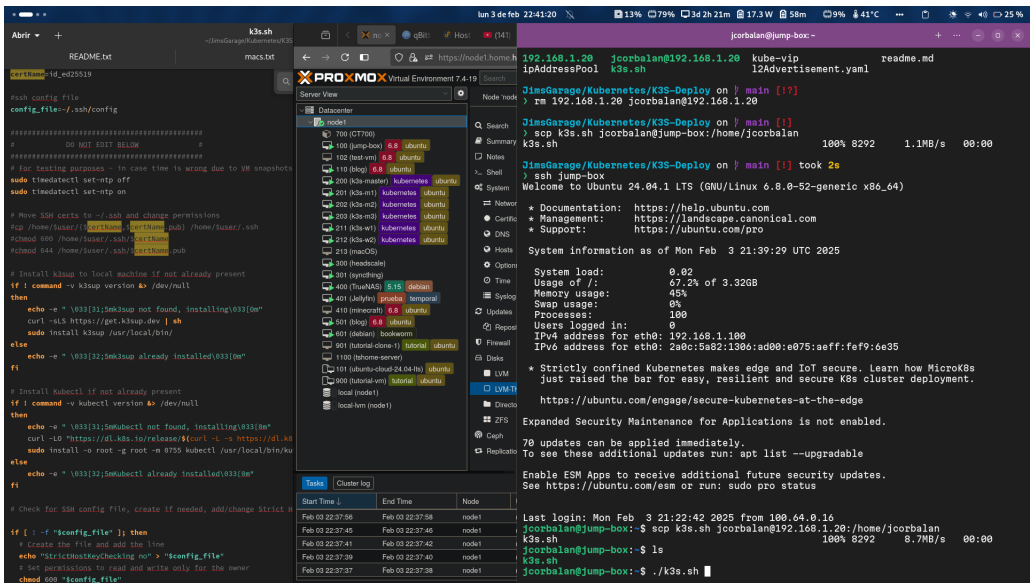


Figura 3.2: Scripts y máquinas virtuales con K3s corriendo el nodo de Proxmox, junto con proyectos personales.

cumplía con las necesidades del departamento. También se experimentó con almacenamiento persistente dentro del clúster, pero debido a que el tamaño de los datos con los que trabaja el departamento son de gran tamaño, se decidió pivotar a un almacenamiento en red.

Más adelante se experimentó con Talos Linux, tras tener dificultades ejecutando los *scripts* y recreando el sistema en el equipo del laboratorio, debido a la naturaleza mutable de Ubuntu, y su poca preparación para la ejecución de Kubernetes. Finalmente se decidió utilizar Talos Linux.

3.2 Instalación en el laboratorio

Instalé Proxmox en el equipo, siguiendo una instalación normal, para posteriormente configurar los repositorios gratuitos y actualizar el servidor a la última versión.

Una vez hecho esto, seguí los mismos pasos que había realizado previamente en casa, creando más máquinas virtuales con ubuntu-24.04-server, aplicando los scripts generados. Durante el proceso, tuve varios problemas al desplegar el sistema de Kubernetes por varias constantes del sistema, Debido a esto, finalmente se instaló Talos Linux.

Durante el despliegue, los nodos, debido a la configuración de la red en el laboratiorio, tenían problemas para descargar las imágenes de Talos Linux, como se puede ver en la figura 3.3, por lo que se utilizó la máquina de administración como registro local, como se representa en la figura 3.4. Una vez solucionada esta problemática, K8s quedó desplegado, como se puede ver en la figura 3.5.

Una vez creado el clúster, se clonaron los manifiestos oficiales de Kubeflow (*Kubeflow*

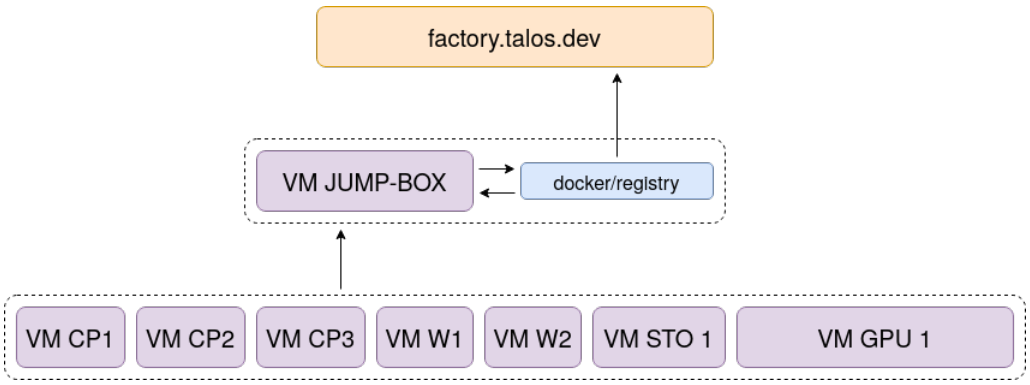


Figura 3.3: Diagrama de utilización del registro local de Docker.

Manifests (2025)),desplegué el sistema en su versión v1.10 con *kubectl apply*. El sistema quedó desplegado, y tras esto se probó su funcionamiento, pero no se pudo continuar debido a exigencias de tiempo, por lo que el equipo se devolvió a su configuración previa a este trabajo.

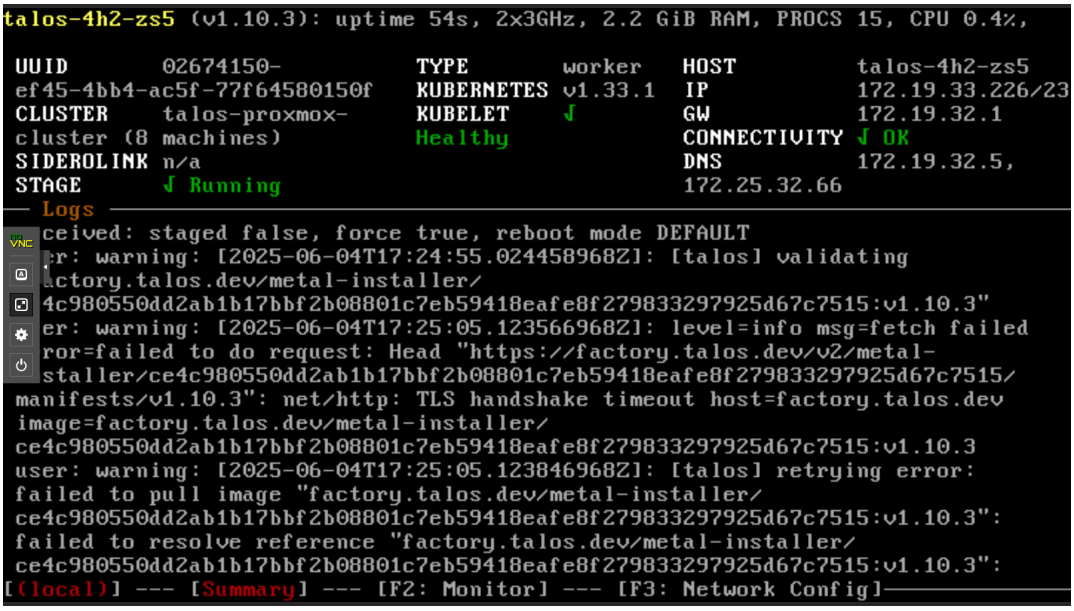


Figura 3.4: Problemas descargando las imágenes de Talos Linux en el laboratorio.

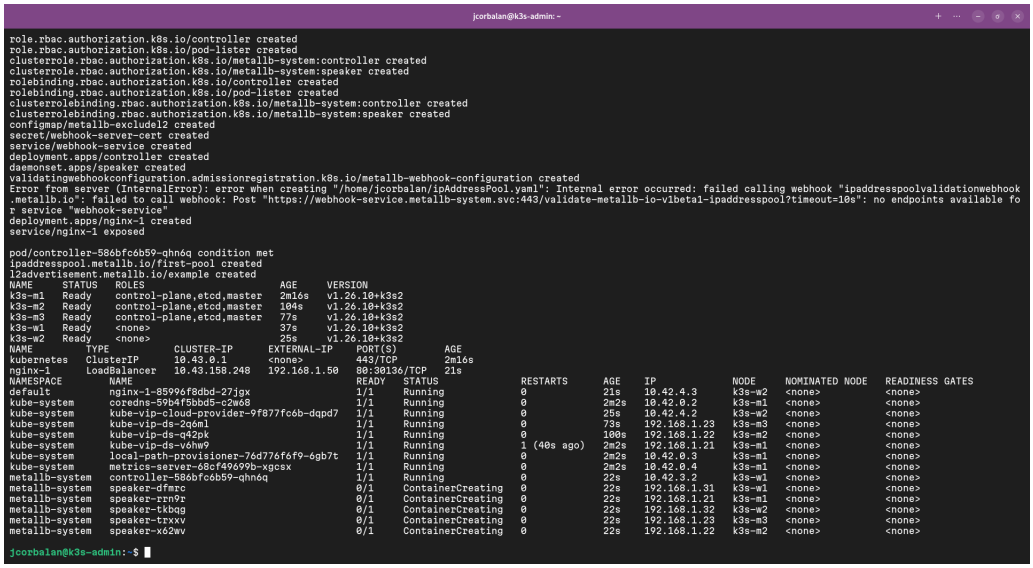


Figura 3.5: El sistema levantado visto desde kubectl.

4 Solucion final y Guía de despiegue

En este apartado se describe las partes de la solución final y los pasos a seguir para poder configurar el sistema al completo. Algunos de estos pasos ya se han seguido en el equipo del laboratorio, pero se indican para que se encuentre todo el proceso documentado.

4.1 Resumen

Debido a las necesidades que tenemos, se ha decidido utilizar un hipervisor de tipo 1. La posibidad de disponer de varias máquinas virtuales nos facilitará el uso de los recursos de *hardware* que disponemos. Se ha escogido *Proxmox Virtual Environment* (Proxmox VE) frente a VMWare ESXi debido a que se trata de un producto de código abierto y a la existencia de un plan de uso gratuito. Además, dispongo adicionalmente de experiencia utilizando y administrando sistemas en este entorno, y existe una gran comunidad detrás.

Para los sistemas operativos que se ejecutarán en las máquinas virtuales, se ha escogido Talos Linux para los nodos donde correrá K8s, y Debian para la máquina virtual desde donde administraremos el clúster. La elección de Talos Linux radica en su simpleza de despliegue, inmutabilidad, sencillez y nivel de abstracción. La elección de Debian para la administración del cluster se basa en su estabilidad y popularidad.

Se crearán un total de cinco máquinas virtuales con Talos Linux, tres de las cuales ejecutarán el plano de control de K8s, y las otras dos utilizarán las dos tarjetas gráficas de las que dispone el equipo. En estos dos últimos nodos se ejecutarán todas las tareas relacionadas con el cómputo del sistema. Todo esto se puede ver de forma visual en la figura 4.1.

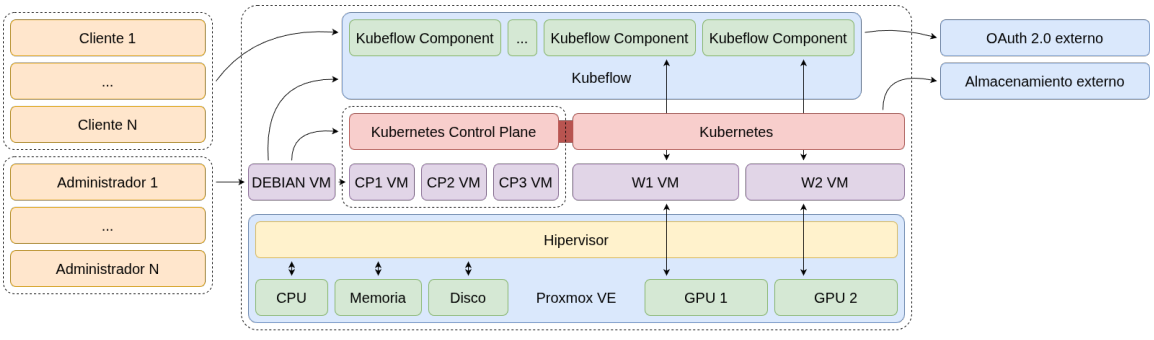


Figura 4.1: Diagrama de la solución final.

Para el almacenamiento persistente del clúster se utilizará un almacenamiento de red basado en NFS, que nos permitirá un almacenamiento rápido, robusto y paralelo. No necesita-

remos una plataforma para la gestion de K8s, ya que dispondremos sólo de un cluster.

Se utilizará Kubeflow como sistema para realizar tareas de MLOps, ya que se trata de una solución robusta y completa. Además, se dispone de manifiestos de despliegue que nos ayudarán a desplegar el sistema, y se pueden modificar para adaptarlo a nuestras necesidades.

Para la autentificación de usuarios, Kubeflow incorpora Dex (*Dex | Documentation* (2025)), que nos permite conectar la autentificación de Kubeflow a cualquier proveedor a través de OpenID. De esta forma podremos utilizar las cuentas de la Universidad de Alicante para autenticarnos.

4.2 Configuración de la BIOS

Lo primero que se debe hacer es comprobar que la BIOS se encuentre actualizada a la última versión, que esté habilitado el perfil *Extreme Memory Profile* (XMP) correcto para la memoria, de tal forma que se aproveche todo el rendimiento de esta, comprobar que se encuentra activada la aceleración por hardware para la virtualización (suele encontrarse como VT-x o AMD-V, en función del fabricante del procesador), y activar la *Input Output Memory Management Unit* (IOMMU), para poder realizar *hardware passtrough*. Como se ha explicado anteriormente, esto permitirá que las máquinas virtuales puedan controlar de forma directa componentes del sistema, en nuestro caso lo utilizaremos para las tarjetas gráficas.

4.3 Instalación de Proxmox Virtual Environment

Proxmox VE dispone de un instalador gráfico, el cual guía al usuario durante todo el proceso. Se ha realizado una instalación normal, tal como se describe en la guía oficial de instalación (*Proxmox VE | Installation* (2025)).

Una vez instalado, debemos introducir en el navegador la url que hemos configurado para nuestro servidor, seguido del puerto de la interfaz de Proxmox VE. En mi caso, sería https://192.168.1.2:8006/.

La primera vez que nos conectemos veremos una advertencia de seguridad, ya que el certificado HTTPs es auto-firmado. Debemos pulsar avanzado, aceptar el riesgo y continuar, ya podremos conectarnos.

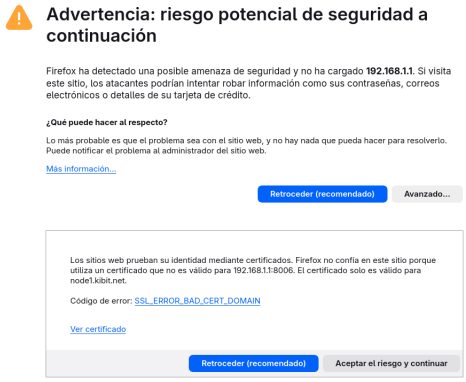


Figura 4.2: Advertencia del navegador al conectar a Proxmox VE.

Veremos un diálogo de autentificación, en la que debemos introducir el usuario root y la contraseña que hemos introducido durante la instalación.

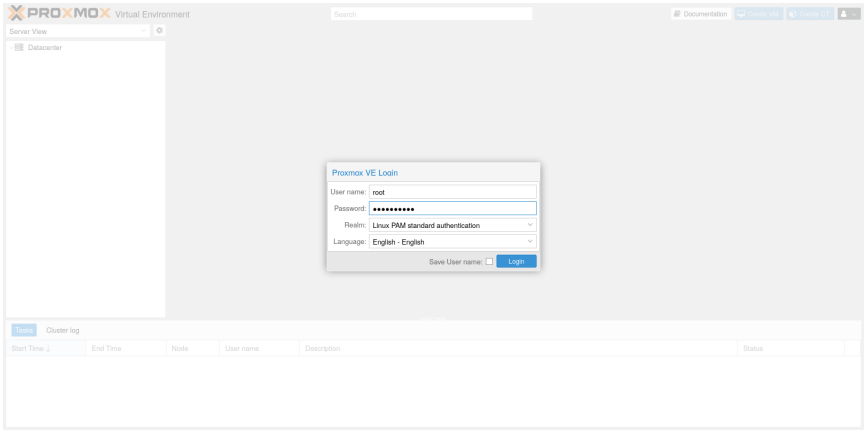


Figura 4.3: Diálogo de autentificación en Proxmox VE.

Una vez accedamos, veremos un aviso de el servidor no tiene licencia activada.

Siempre que no pagemos una licencia de Proxmox VE, aparecerá este mensaje que nos informa que no disponemos de licencia empresarial. Es algo normal, que la compañía pone ahí avisarnos que el sistema puede ser menos estable que la versión empresarial, para atraer clientes. Por último, debemos configurar los repositorios sin suscripción, para que el sistema siga recibiendo actualizaciones.

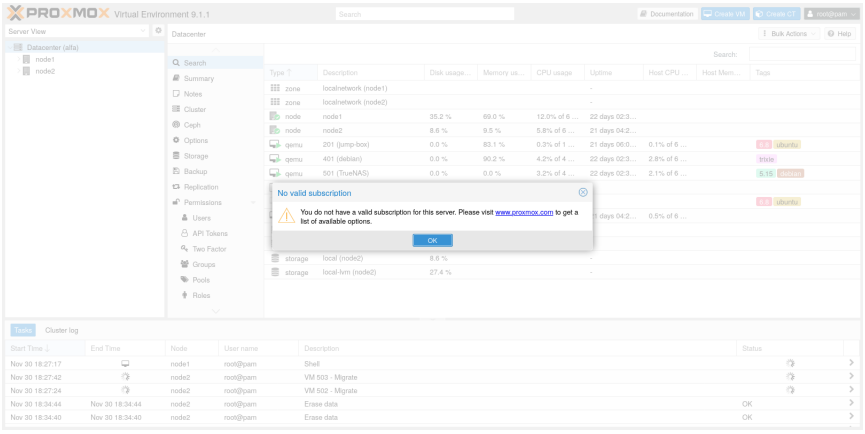


Figura 4.4: Advertencia de suscripción inválida.

Para la realización de este trabajo, he vaciado uno de mis nodos en mi clúster personal de casa, por lo que en mi caso aparecen dos nodos, pero al seguir esta guía sobre el equipo del laboratorio deberá verse sólo uno.

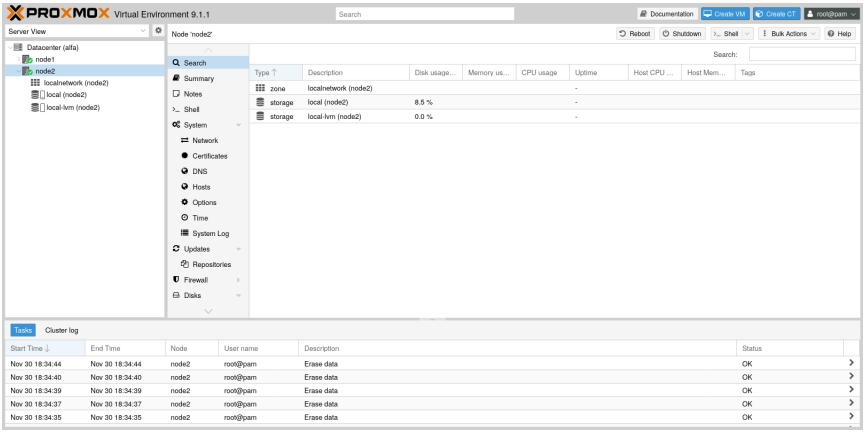


Figura 4.5: Nodo de Proxmox VE vacío.

Ya tenemos nuestro nodo de Proxmox VE listo para trabajar.

4.4 Creación y configuración de las máquinas virtuales

Crearemos un total de 7 máquinas virtuales. La primera, utilizará debian y será la máquina desde la que administraremos el cluster. La segunda, será un NAS virtualizado, en este caso TrueNAS. Esta máquina virtual no se debe implementar en el entorno de producción, solo se ha creado para poder realizar esta guía. Las otras 5, se utilizarán para Kubernetes, siendo 3 nodos master y 2 workers.

4.4.1 Instalación de Debian en la máquina que administrará el clúster

El primer paso para instalar Debian es conseguir el ISO de instalación. Para ello, vamos a la página de descarga, https://www.debian.org/distrib/, y haremos click derecho sobre iso netinst para PC de 64 bits, copiar al portapapeles.

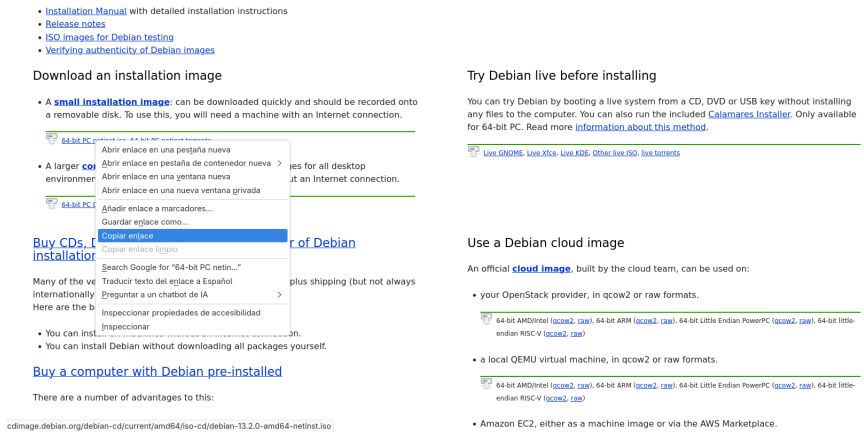


Figura 4.6: Menú desplegable mostrado con el click derecho

Después, debemos volver a nuestra interfaz de Proxmox VE, ir al almacenamiento local y a la pestaña de ISO images, donde pulsaremos el botón Download from URL.

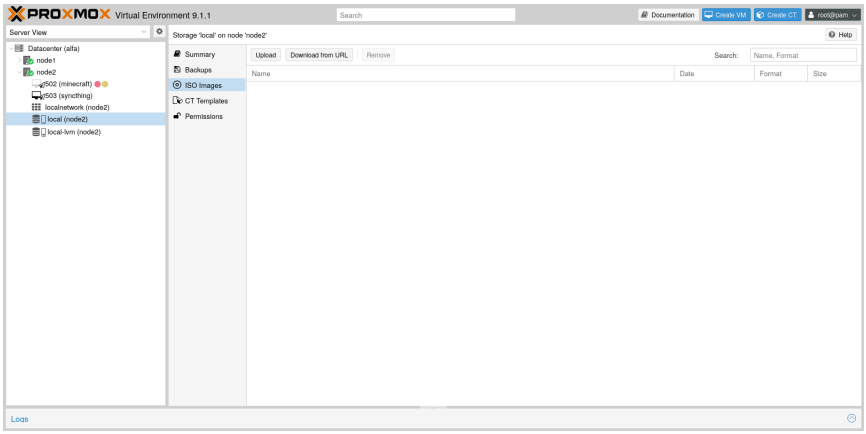


Figura 4.7: Pantalla que muestra los ISOs que tenemos descargados

Cuando lo pulsemos, se mostrará un menú que nos permitirá descargar la imagen. Debemos introducir la URL que hemos copiado en el campo URL, y pulsar el botón Query URL. Si la URL es válida, obtendremos algunos metadatos de la imagen, y podremos comenzar la descarga con el botón Download.

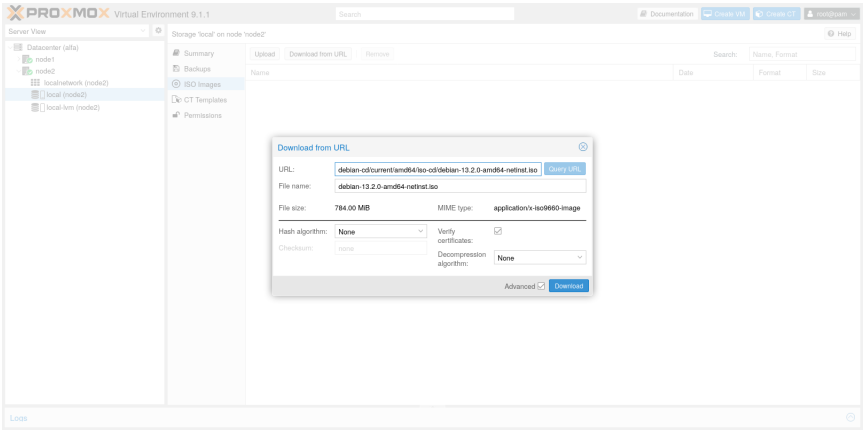


Figura 4.8: Descarga del ISO de Debian desde URL

Una vez pulsado, comenzará la descarga. Aunque se puede realizar en segundo plano, esperaremos a que termine la descarga y muestre TASK OK, para verificar que lo hemos realizado correctamente antes de continuar.

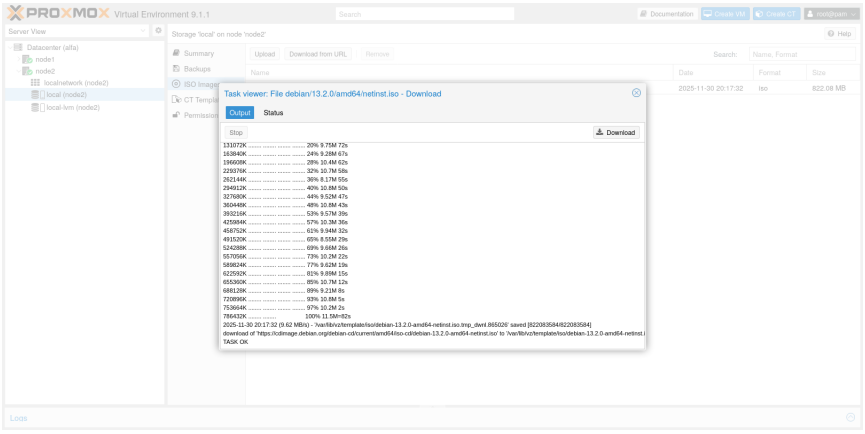


Figura 4.9: Progreso de la descarga del ISO de Debian

Ahora, procederemos a crear la máquina virtual. Sobre nuestro nodo, en la barra lateral, debemos hacer click derecho, y pulsar sobre Create VM.

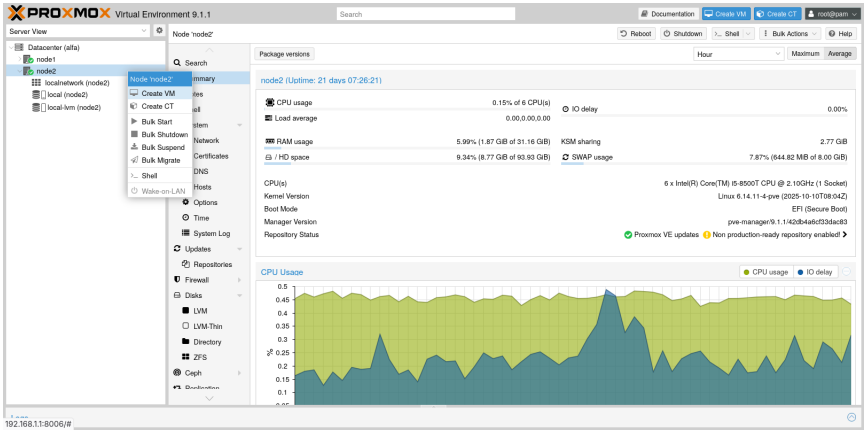


Figura 4.10: Creación de una nueva máquina virtual

Primero le daremos un nombre que nos ayudará a identificar nuestra máquina, junto con un ID, en este caso yo he dejado el por defecto, 100. Iremos avanzando las pantallas, donde, respecto a los predeterminados, marcaremos la casilla para el agente QEMU y aumentaremos el número de núcleos a 2.

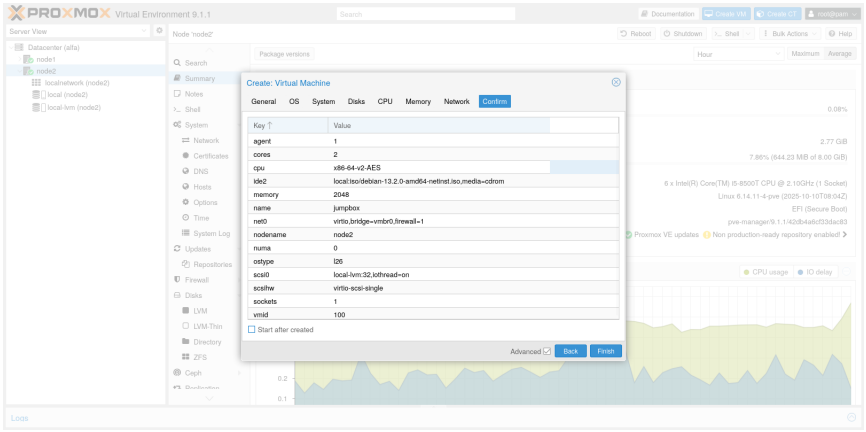


Figura 4.11: Resumen de los ajustes seleccionados en la máquina virtual Debian

Una vez creada la máquina virtual, pulsamos sobre ella, y luego pulsaremos Start Now. Una vez arranque, seleccionaremos Graphical Install.

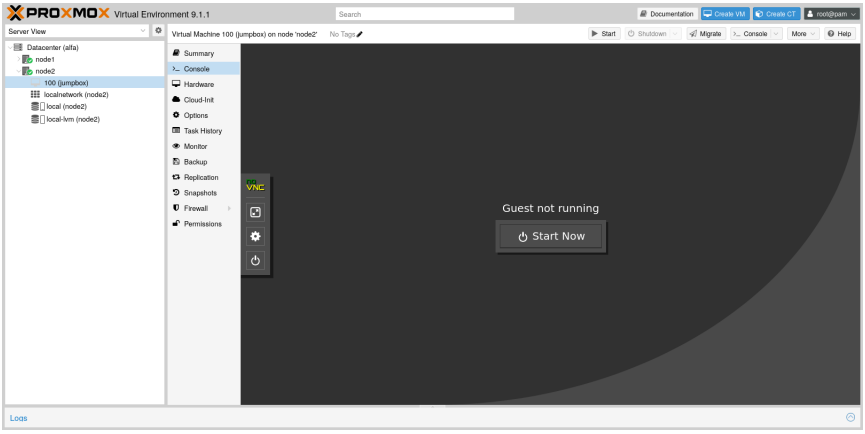


Figura 4.12: Máquina virtual Debian sin arrancar

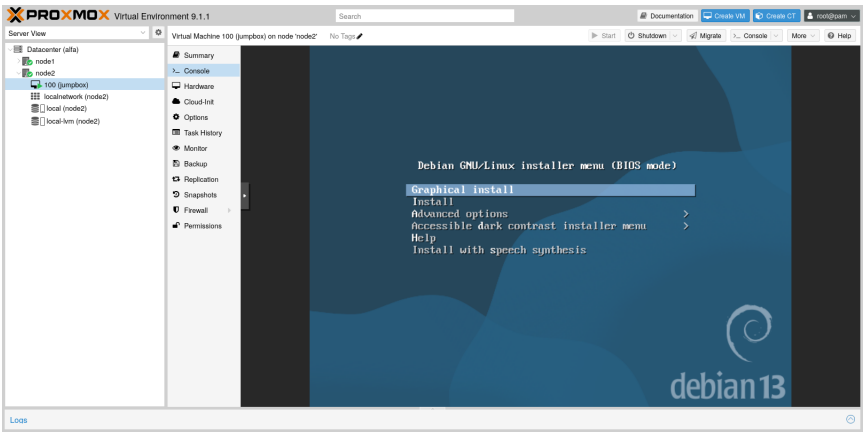


Figura 4.13: Menú inicial del instalador de Debian

Escogeremos el idioma, la ubicación, el lenguaje del teclado, damos el nombre de la máquina, contraseña del usuario root, y luego el usuario principal, que hemos llamado Administrador, con nombre de usuario administrador, introducimos su contraseña, configuramos el reloj, particionamos el disco, configuramos el gestor de paquetes, y desmarcamos el entorno gráfico de Debian y el escritorio GNOME, y habilitamos el servicio SSH. Instalamos GRUB y finalizamos la instalación.

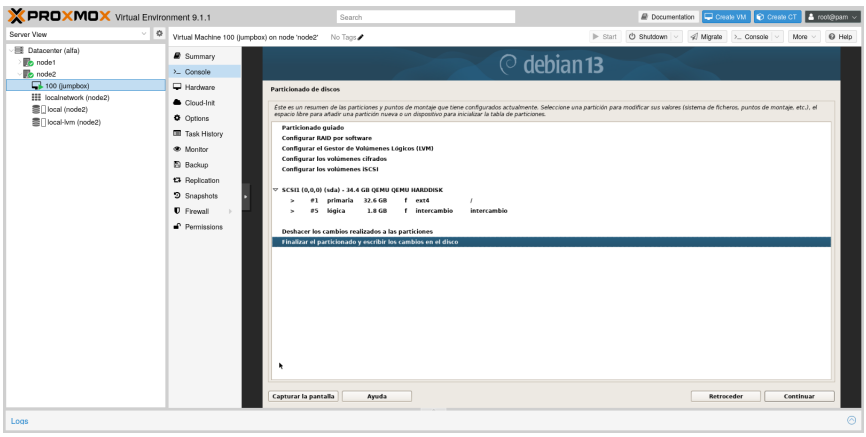


Figura 4.14: Resumem del particionado de discos en el instalador Debian

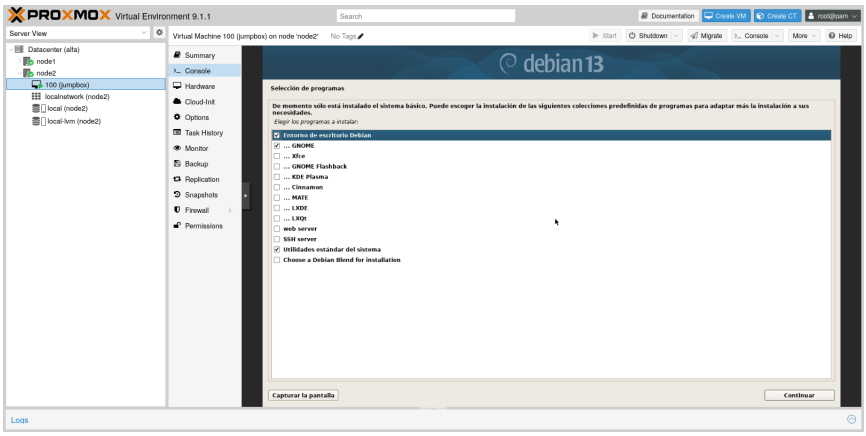


Figura 4.15: Selección de programas en el instalador de Debian

Desde la pestaña de Hardware en Proxmox VE, editamos el CD/DVD Drive y eliminamos el ISO, marcando Do not use any media. Al arrancar de nuevo la máquina virtual ya tendríamos el sistema instalado y listo para su configuración.

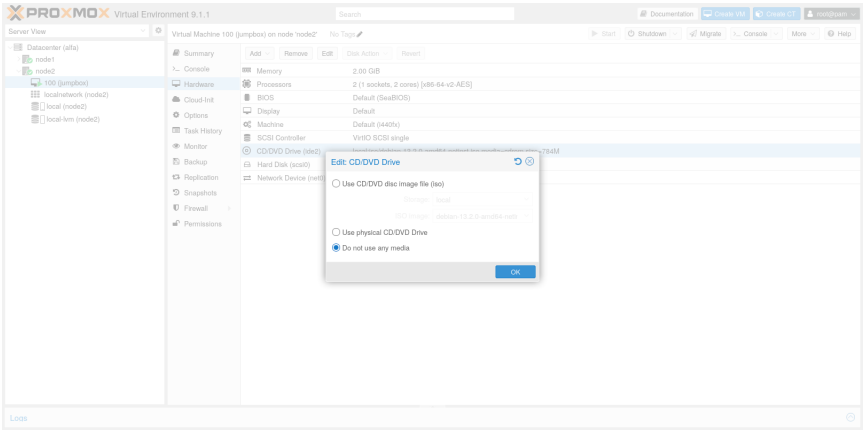


Figura 4.16: Eliminación del ISO de instalación en la máquina virtual Debian

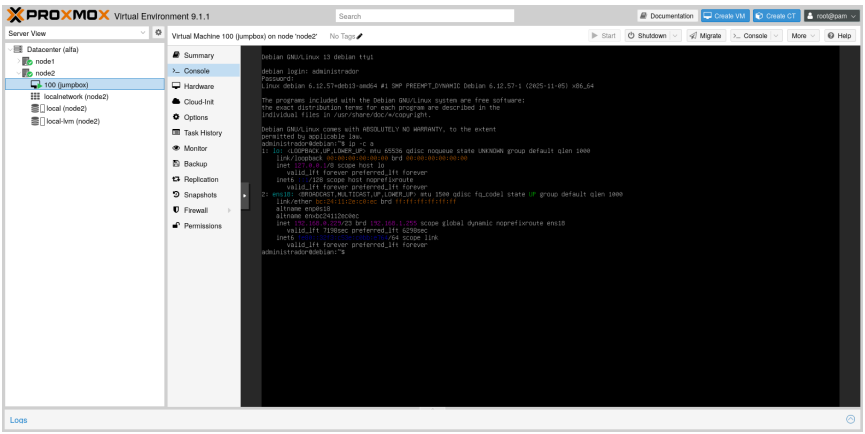


Figura 4.17: Primer logueo en la máquina virtual Debian

Una vez hecho esto, ya podemos conectarnos a través de ssh para configurar el sistema. Esta misma configuración se puede realizar a través de la consola que vemos en Proxmox, pero trabajar por ssh siempre es más cómodo.

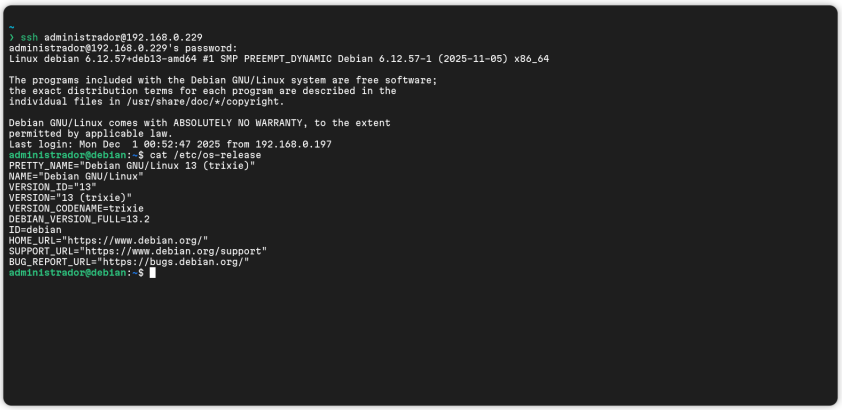


Figura 4.18: Primera conexión ssh a la máquina virtual Debian

Luego se ha instalado sudo, y dado permisos al usuario Administrador.



Figura 4.19: Instalación y configuración de sudo en la máquina virtual Debian

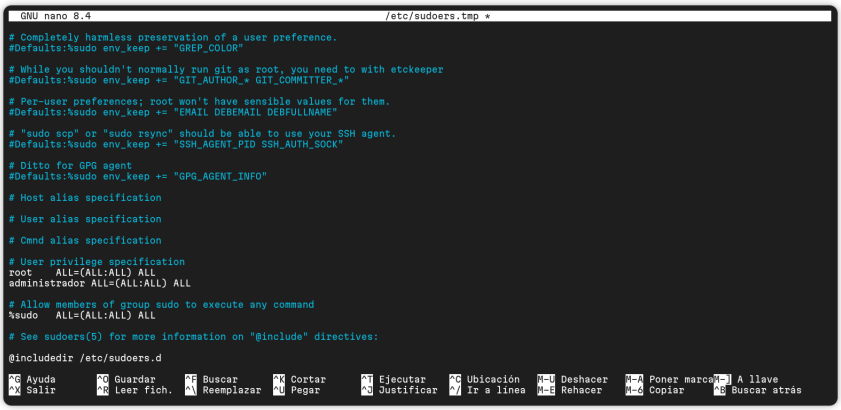


Figura 4.20: Modificación realizada con visudo

Después, instalamos curl, talosctl, kustomize, kubectl, helm y git.

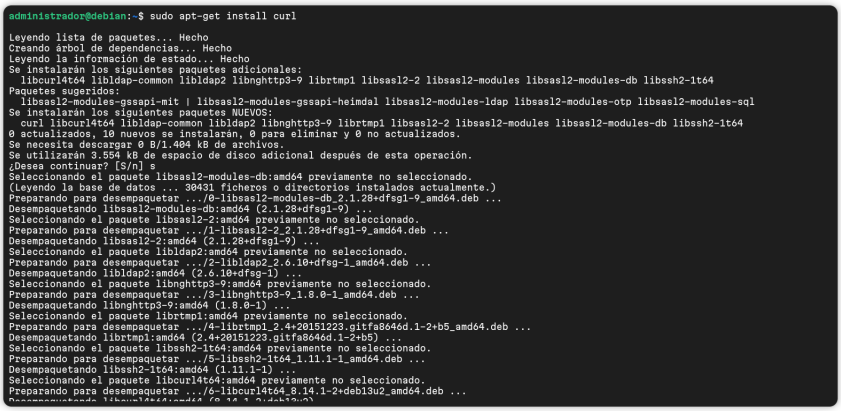


Figura 4.21: Instalación de curl

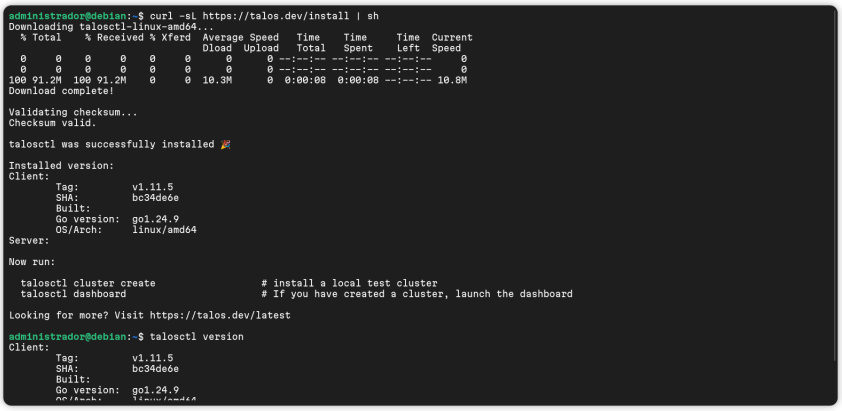


Figura 4.22: Instalación de talosctl

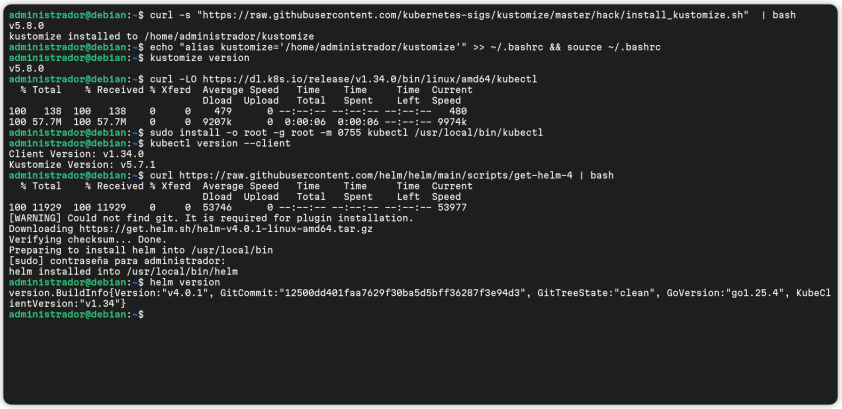


Figura 4.23: Instalación de kustomice, kubectl y helm

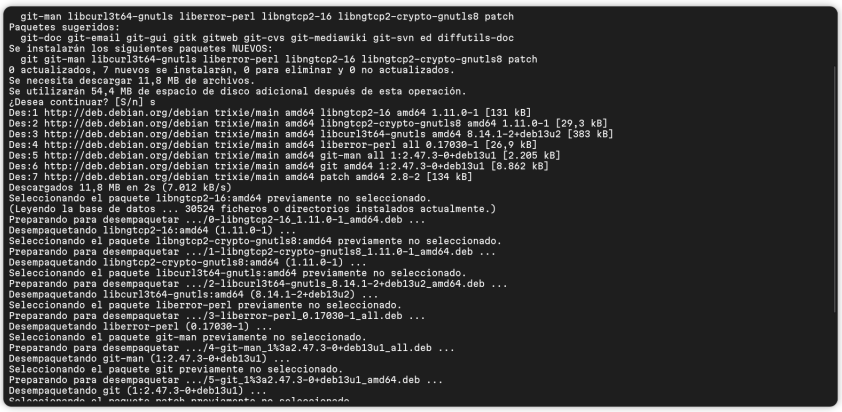


Figura 4.24: Instalación de git

Una vez realizado esto, ya tendríamos lista nuestra máquina virtual debian.

4.4.2 Instalación de Talos OS en el resto de máquinas virtuales

De nuevo, el primer paso para instalar Talos Linux es conseguir el ISO de instalación, aunque este se obtiene de forma diferente a Debian. En el caso de Talos Linux, como se trata de una distribución diseñada específicamente para Kubernetes, el número de paquetes y extensiones es mínimo, por lo que debemos de manualmente seleccionar lo que nuestro ISO necesitará. Si nos faltara alguna extensión, siempre pueden añadirse más tarde.

Para ello, vamos a la página de descarga, https://factory.talos.dev/ y configuramos el ISO a nuestro gusto.

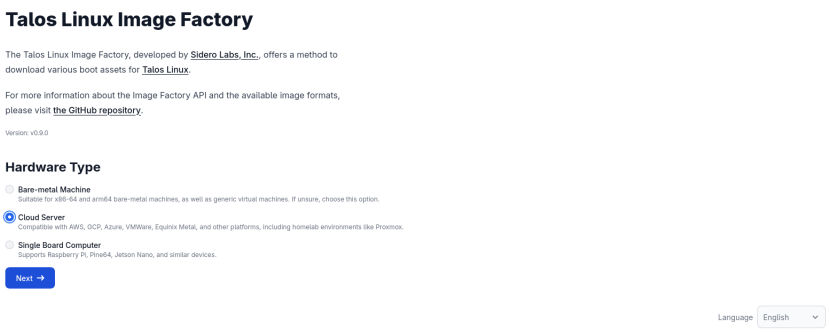


Figura 4.25: Factoría de imágenes de Talos Linux

Una vez completado el proceso, acedemos a la interfaz web en la dirección IP que tenga el equipo, en el puerto de Proxmox VE (8006). En mi caso, sería la dirección https://192.168.1.2:8006/.

Aquí también veremos una advertencia de seguridad, ya que el certificado HTTPs es autofirmado. Debemos pulsar avanzado, aceptar el riesgo y continuar, ya podremos conectarnos.

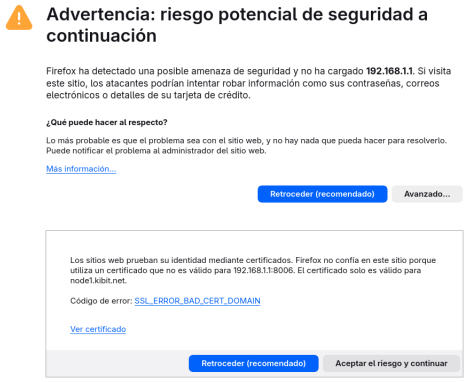


Figura 4.26: Advertencia del navegador.

Veremos un diálogo de autentificación, en la que debemos introducir el usuario root y la contraseña que hemos introducido durante la instalación.

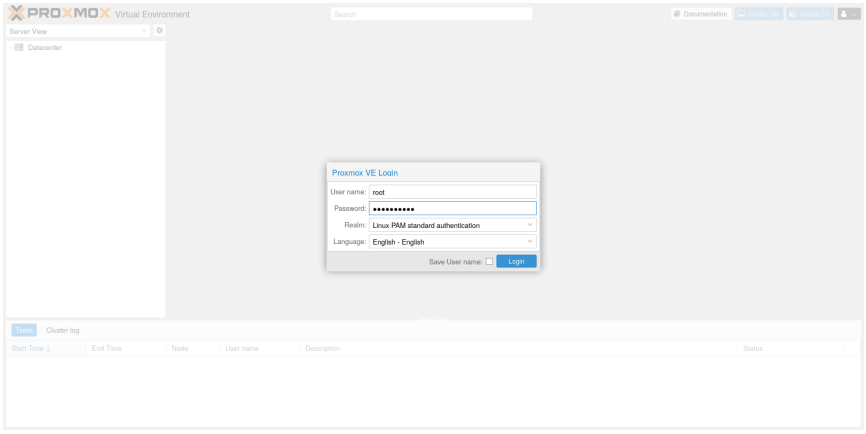


Figura 4.27: Diálogo de autentificación.

eso, este sería el resultado para las opciones Cloud Server, No cloud, amd64, y la extensión para QEMU.

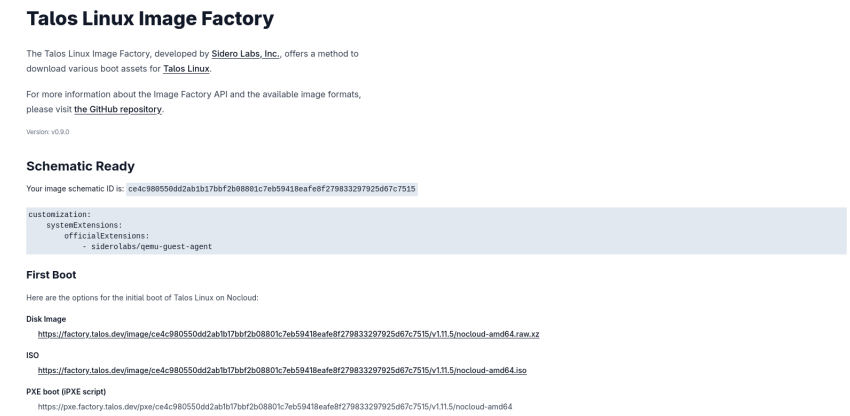


Figura 4.28: Opciones escogidas en la factoría de imágenes de Talos Linux

Ahora debemos copiar la URL para el ISO que aparece, y ir a Proxmox VE para descargarlo, como ya hemos realizado anteriormente. Yo he modificado el nombre del archivo, para que referencie a la versión y opciones escogidas.

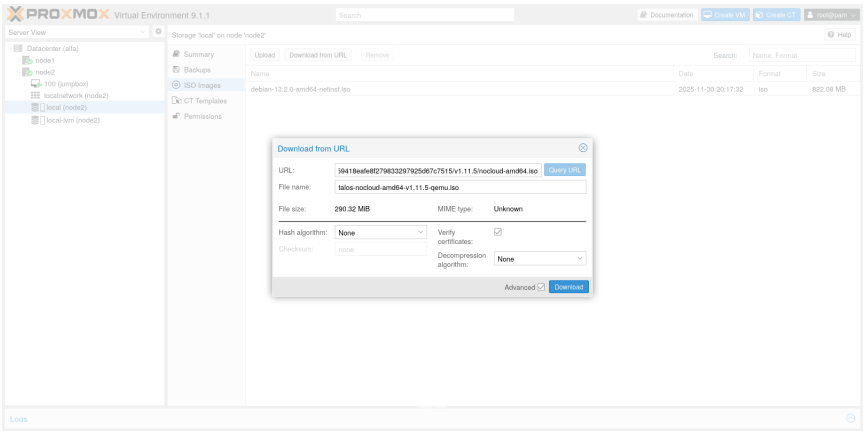


Figura 4.29: Descarga del ISO de Talos Linux desde URL

Ahora, procederemos a crear las máquinas virtuales, de la misma manera que se ha hecho para Debian, pero con otras ocpiones. Crearemos tres máquinas que denominaremos masters (maestros) y dos workers (trabajadores).

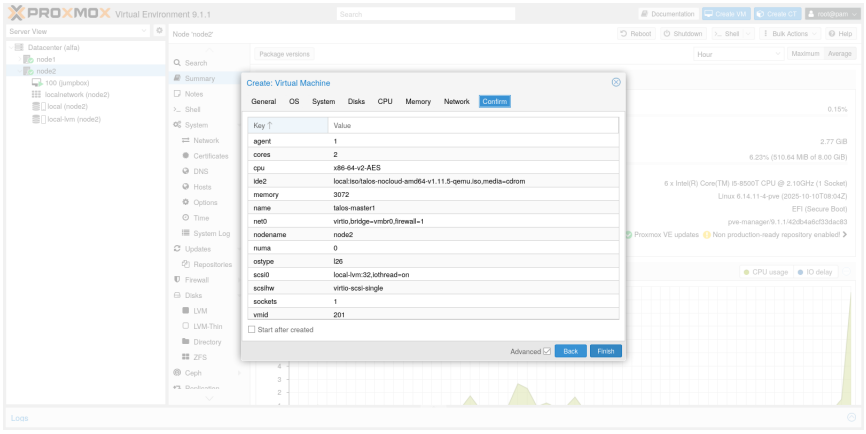


Figura 4.30: Opciones elegidas para los nodos maestros

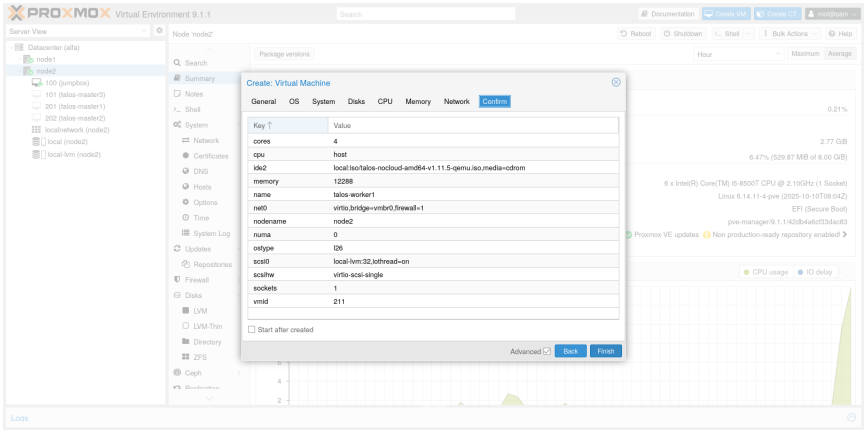


Figura 4.31: Opciones elegidas para los nodos trabajadores

En el caso del procesador de los trabajadores, se ha escogido host, ya que para la ejecución de TensorFlow eran necesarias más instrucciones de las que proporciona el procesador virtualizado x86-64-v2-AES. Destacar que a la hora de desplegar el sistema en el servidor del laboratorio, sería necesario instalar en los nodos trabajadores ciertas extensiones de *drivers* de nvidia, así como hacer el *passtrough* de las líneas PCI que conectan a las tarjetas gráficas directamente a los nodos trabajadores (Una a cada nodo).

Una vez hecho esto, arrancamos las máquinas virtuales, y se instalará automáticamente Talos Linux. Dejaremos los ISOs montados, para poder restablecer un nodo en el arranque siempre que queramos.

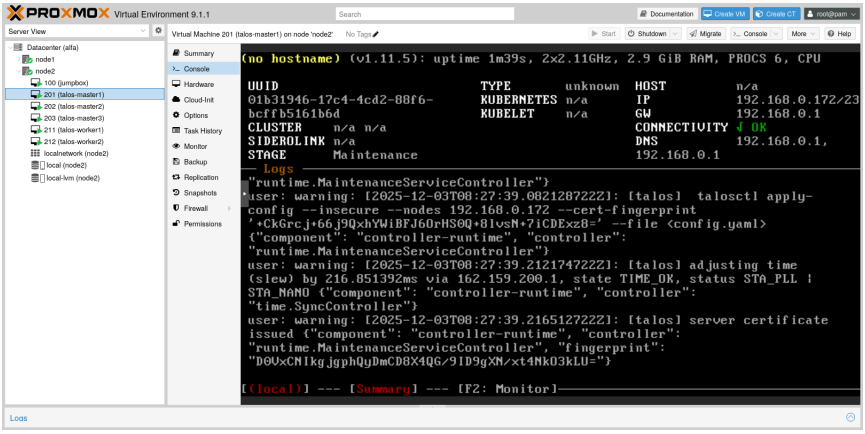


Figura 4.32: Nodo de Talos Linux arrancado, a la espera de ser configurado

4.4.3 Instalación de TrueNAS SCALE

Para la realización de este trabajo, utilizaré un NAS virtualizado. Esto en un entorno de producción no sería recomendado. Lo ideal sería utilizar almacenamiento de red separado a nivel físico. Si esto fuera imposible, al menos se debería utilizar un almacenamiento físico (HHD, SSD o NVME) conectado a la máquina virtual con *pci passtrough*, y no uno virtualizado, como realizaré yo. En mi caso, he escogido TrueNAS SCALE para la realización del prototipado.

De la misma forma que en las anteriores, lo primero es obtener el ISO. Accedemos a la página de descarga oficial, https://www.truenas.com/download-truenas-community-edition/, hacemos click derecho sobre el botón download stable, y copiamos la url de descarga.

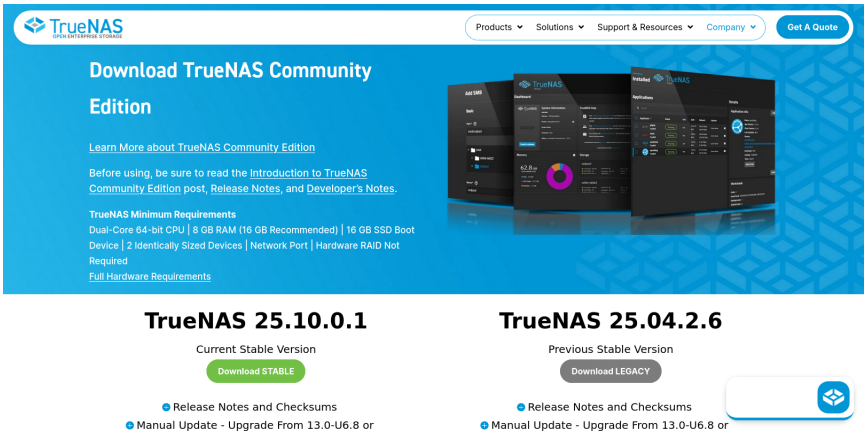


Figura 4.33: Página de descarga del ISO de TrueNAS

Una vez hecho esto, debemos hacer click derecho sobre el botón de descarga de la versión estable, y ir a Proxmox VE para descargarlo.

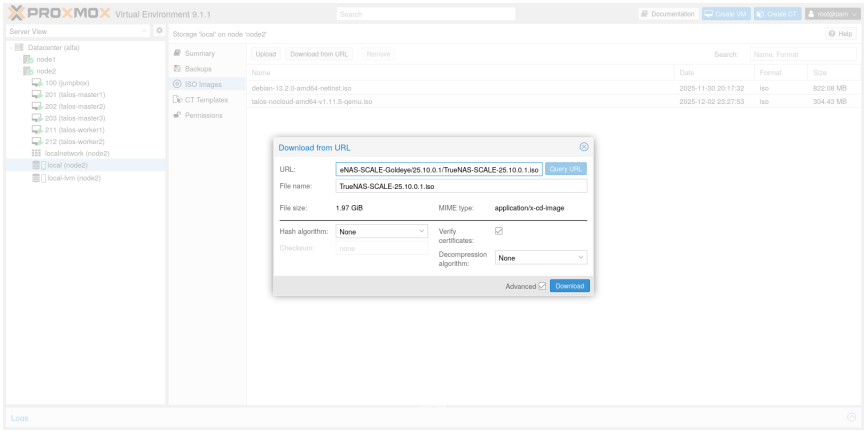


Figura 4.34: Descarga del ISO de TrueNAS desde URL

Ahora, procederemos a crear la máquina virtual, de la misma manera que ya hemos varias veces, pero de nuevo con otras ocpiones. Utilizaremos dos discos, uno que albergará el sistema operativo y otro donde se almacenarán los datos.

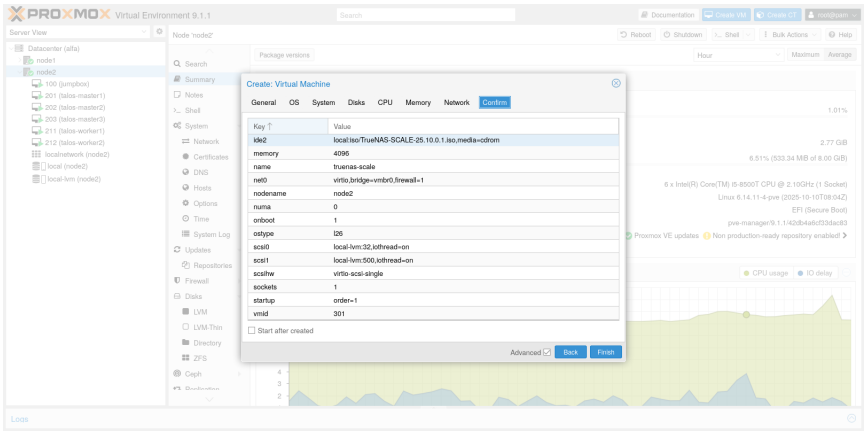


Figura 4.35: Opciones elegidas para la máquina virtual TrueNAS

Una vez creados los discos, para evitar problemas a la hora de crear la *pool* de datos en Truenas, debemos darle un número de serie al disco donde almacenaremos los datos. Esto es algo que todavía no se puede hacer a través de la interfaz gráfica de Proxmox VE, por lo que debemos conectarnos por ssh con la cuenta root. Utilizaremos la misma contraseña que en la interfaz web.

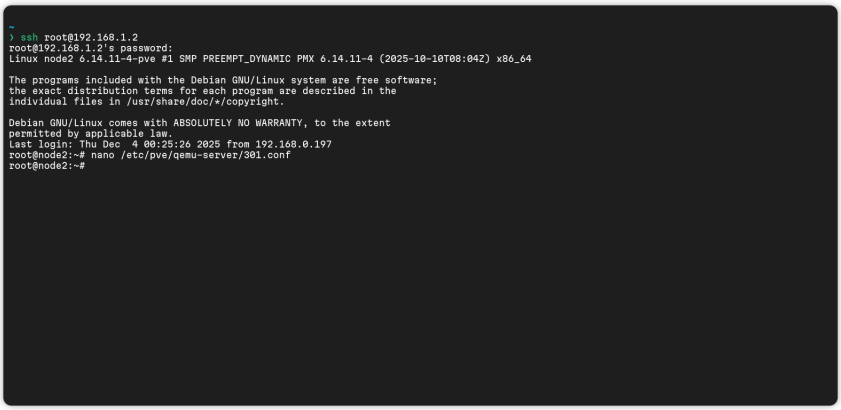


Figura 4.36: Comandos utilizados para indicar un número de serie al disco

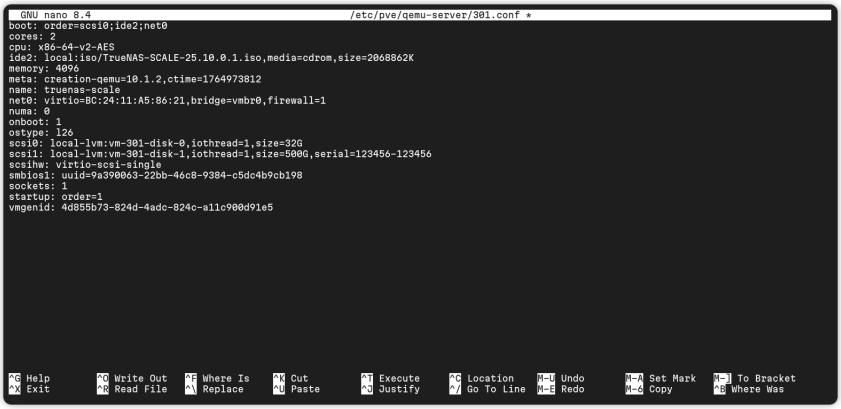


Figura 4.37: Modificación realizada en 301.conf

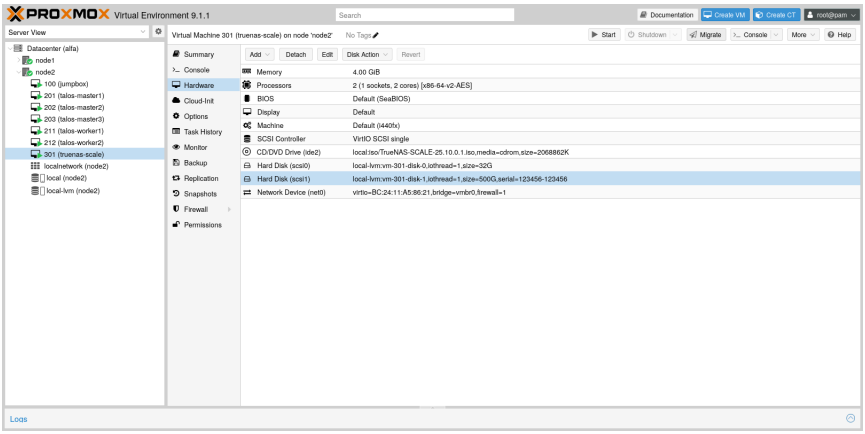


Figura 4.38: Vista del disco y su número de serie desde Proxmox VE

Una vez hecho esto, arrancamos la máquina virtual y comenzamos la instalación. Escogemos *Start TrueNAS SCALE Instalation*, luego *Install/Upgrade*, marcamos el disco de 32 GiB para instalar el sistema operativo,

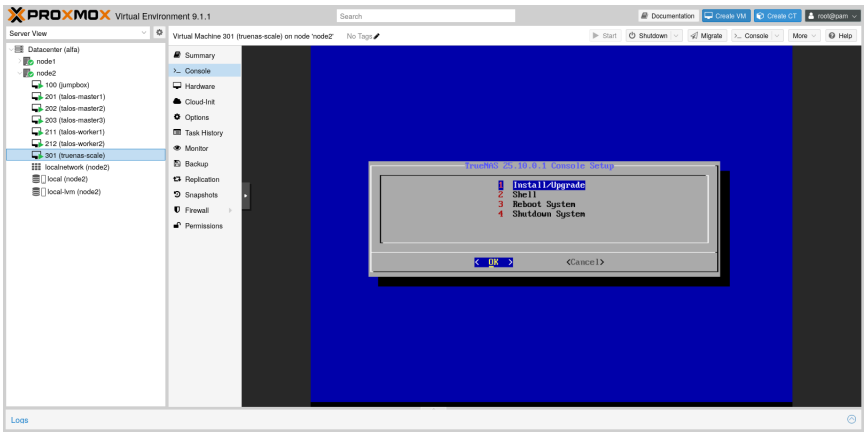


Figura 4.39: Menú inicial del instalador de TrueNAS

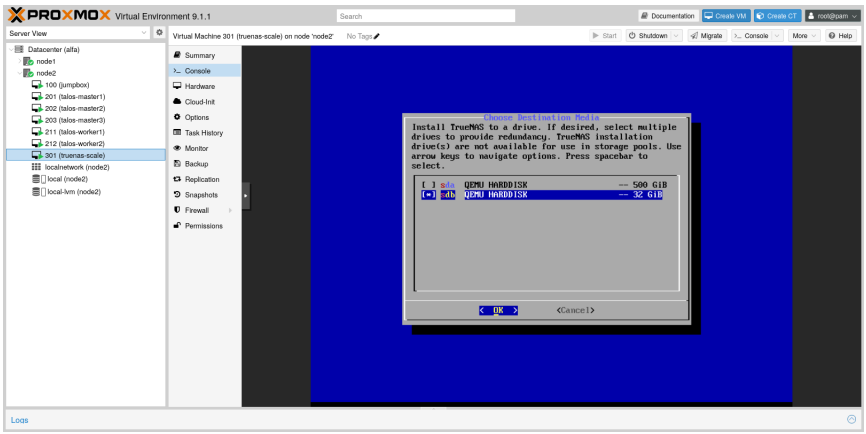


Figura 4.40: Menú para escoger los discos a utilizar durante la instalación de TrueNAS

Después, configuramos la contraseña del usuario administrador.

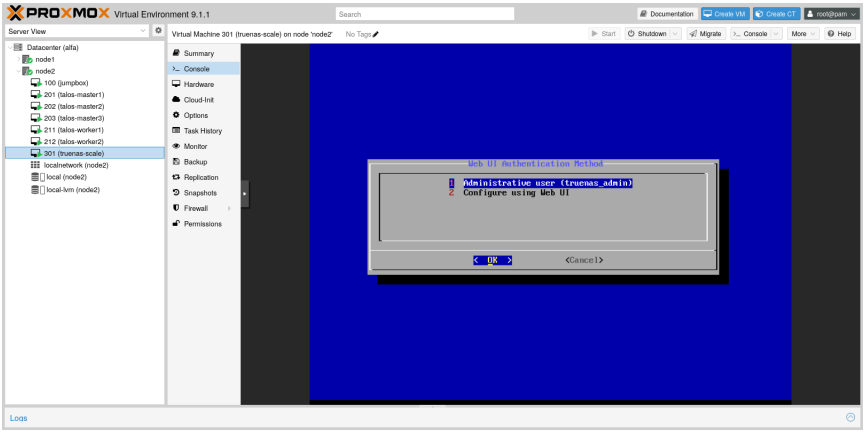


Figura 4.41: Configuración del usuario administrador durante la instalación de TrueNAS

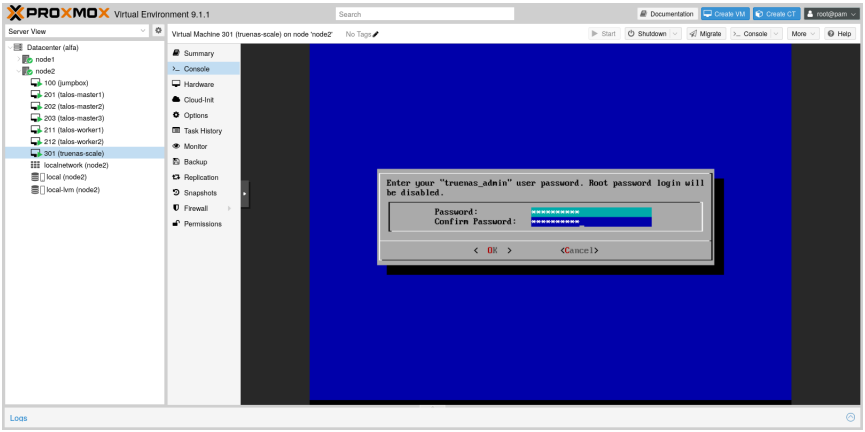


Figura 4.42: Configuración de la contraseña del administrador durante la instalación de TrueNAS

Permitimos el arranque con EFI, y habremos terminado la instalación.

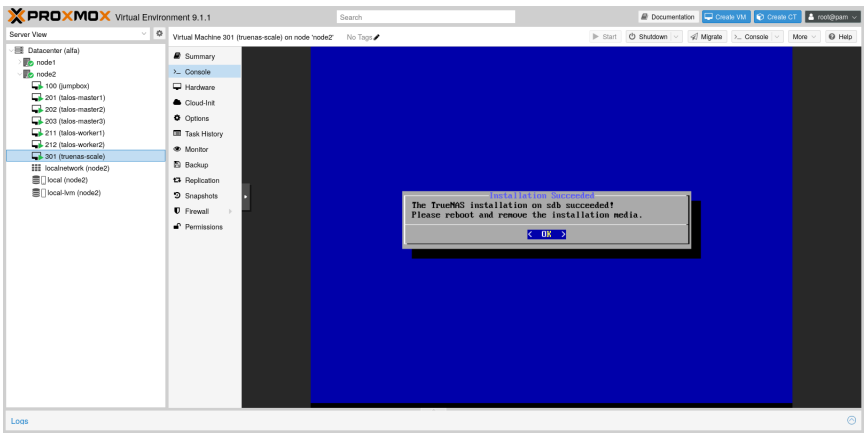


Figura 4.43: Final de la instalación de TrueNAS

Desconectamos el ISO y reiniciamos la máquina virtual.

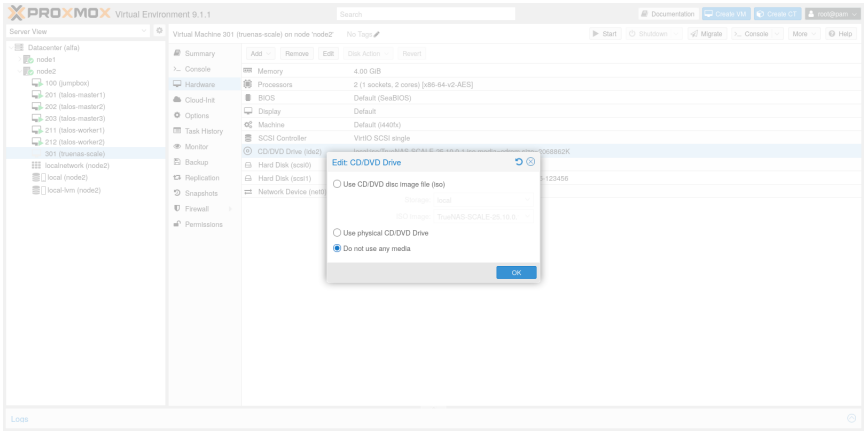


Figura 4.44: Desconexión del ISO en la máquina virtual TrueNAS

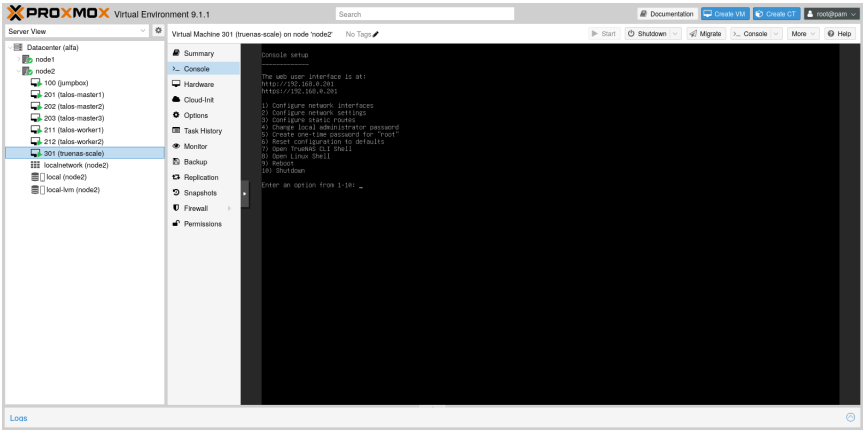


Figura 4.45: Consola de la máquina virtual TrueNAS tras el arranque

Una vez ya hemos reiniciado la máquina virtual, podemos acceder a la interfaz web en la dirección IP que aparece en la consola. En mi caso, sería https://192.168.0.201:8006/.

La primera vez que nos conectemos veremos una advertencia de seguridad, ya que el certificado HTTPs es auto-firmado. Debemos pulsar avanzado, aceptar el riesgo y continuar, ya podremos conectarnos.



Figura 4.46: Advertencia del navegador al conectar a TrueNAS SCALE.

Veremos un diálogo de autentificación, en la que debemos introducir el usuario *truenas_admin* y la contraseña que hemos introducido durante la instalación. Cuando nos conectemos, veremos el *dashboard* de nuestro NAS.

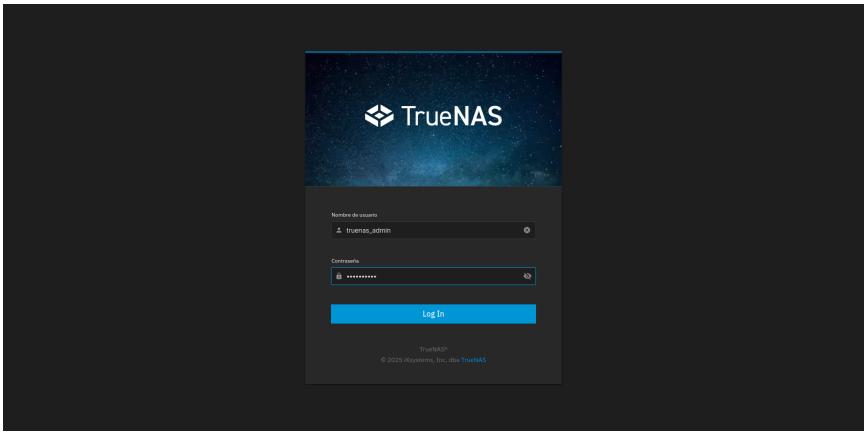


Figura 4.47: Diálogo de autentificación en TrueNAS SCALE.

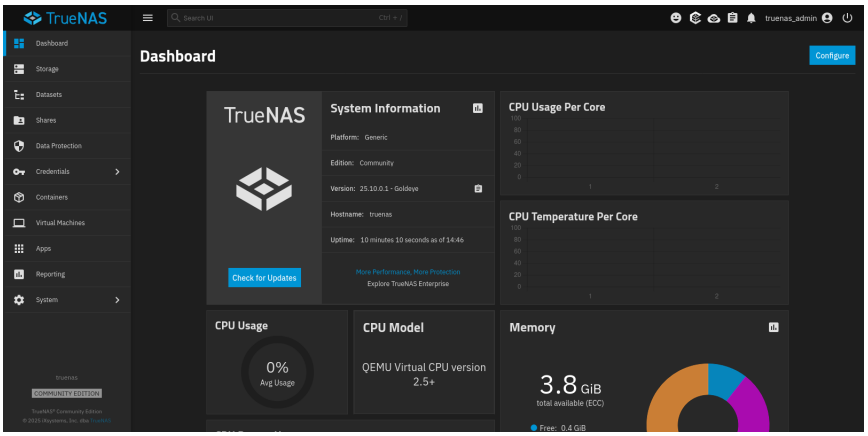


Figura 4.48: Dashboard de TrueNAS SCALE.

Debemos acceder a la pestaña *Storage*, y pulsar sobre *Create Pool*. Introducimos un nombre, seleccionamos el disco de 500 GiB y la topología *STRIPE* (ya que sólo tenemos un disco y estamos realizando un prototipado) y pulsamos sobre *Create Pool*.

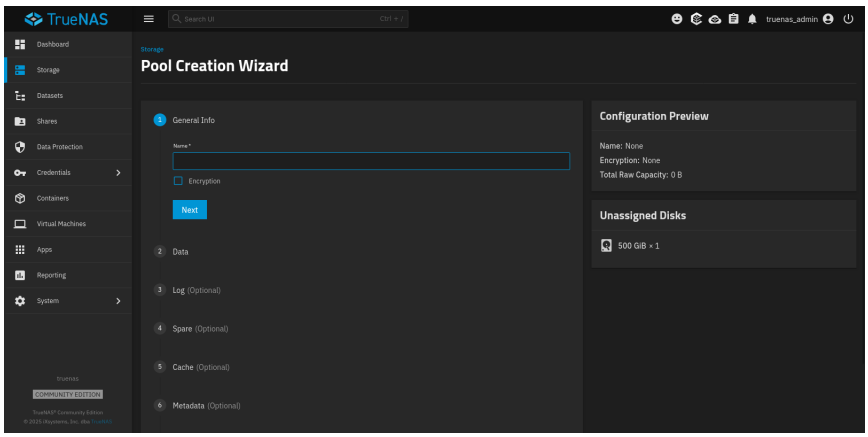


Figura 4.49: Asistente de creación de *pools* en TrueNAS SCALE.

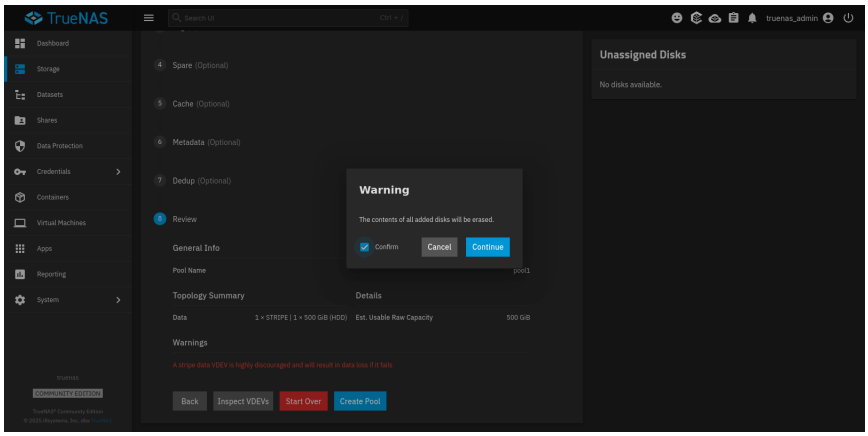


Figura 4.50: Creación de la *pool* en TrueNAS SCALE.

Una vez esté creada la *pool* de datos, vamos a la pestaña *Shares* y pulsamos sobre *Add* en *UNIX (NFS) Shares*. Ahí debemos introducir la ruta a la carpeta que queremos compartir en red, y una descripción.

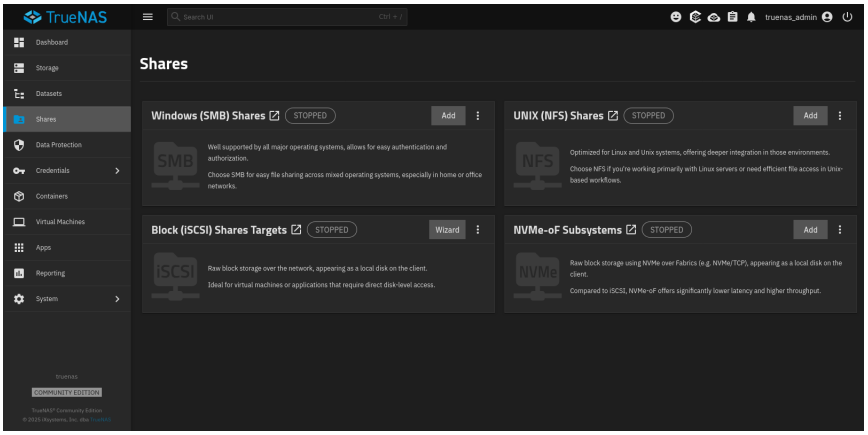


Figura 4.51: Pestaña *Shares* en TrueNAS SCALE.

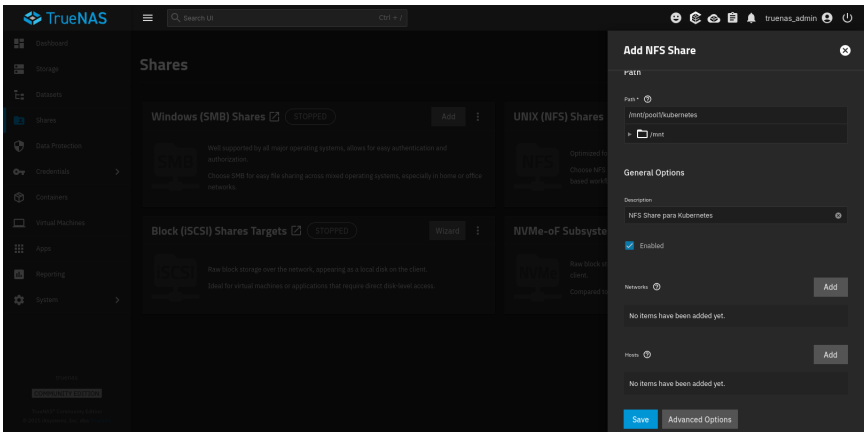


Figura 4.52: Creación de una *share* NFS en TrueNAS SCALE.

En opciones avanzadas, mapear todos los usuarios que se conecten a root, para que puedan escribir en la partición de red.

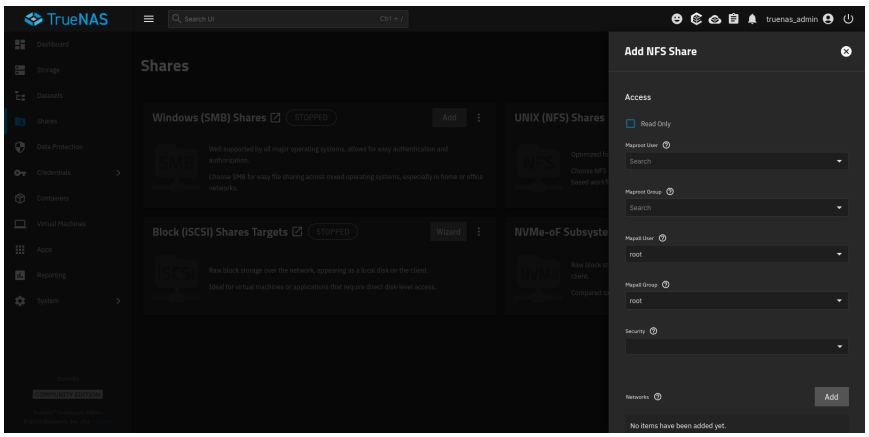


Figura 4.53: Mapeado de usuarios en una *share* NFS en TrueNAS SCALE.

Por último aceptamos el arranque automático del servicio NFS.

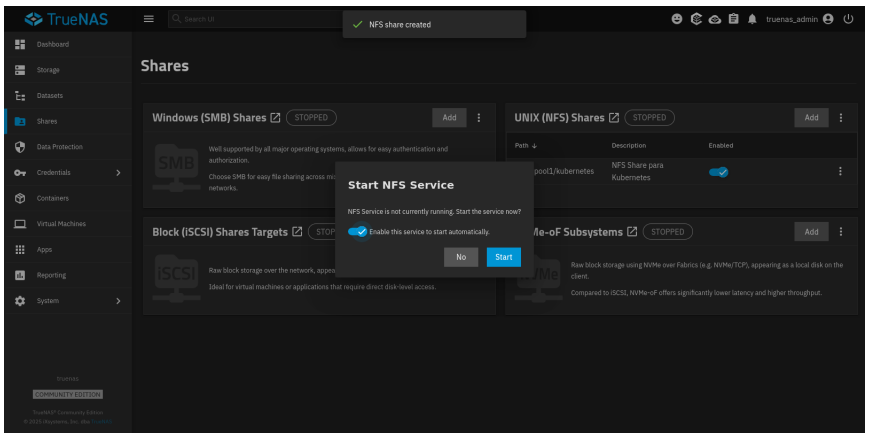


Figura 4.54: Diálogo automático para activar el servicio NFS en TrueNAS SCALE.

4.5 Levantamiento del clúster

Para levantar el clúster, una vez tengamos los nodos arrancados, desde la máquina debian, anotamos la dirección IP de uno de los nodos maestros, y ejecutamos *talos gen config*, apuntando al nodo maestro, del cual hemos anotado la IP, y con el *schematic* ID que hemos obtenido en la fábrica de imágenes. Esto generará varios ficheros en la carpeta *_out*: *controlplane.yaml*, *worker.yaml*, y *talosconfig*.

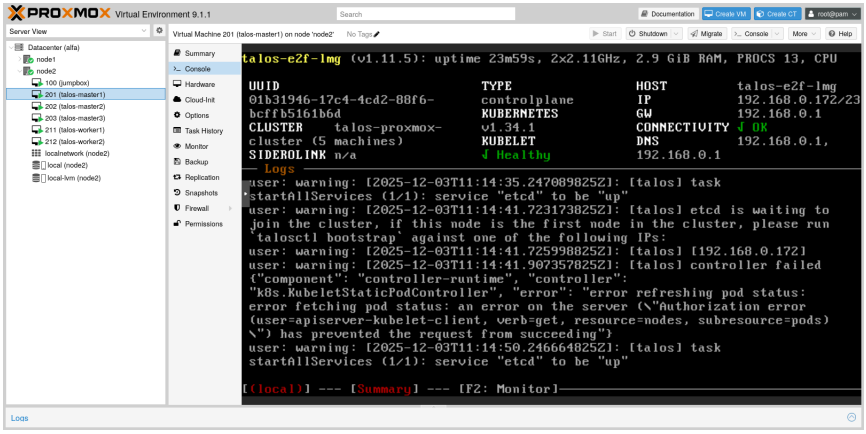


Figura 4.55: Nodo esperando a que se realice el bootstrap del servidor

Una vez hecho esto, aplicamos la configuración a cada uno de los nodos, que se irán reiniciando y uniendo al cluster. Configuramos el *endpoint* y uno de los *workers* y realizamos el *bootstrap* del clúster. Añadimos el resto de nodos a *talosconfig*. Por último, recuperamos el fichero *kubeconfig* y lo movemos a *.kube/config* para que *kubectl* pueda encontrarlo.

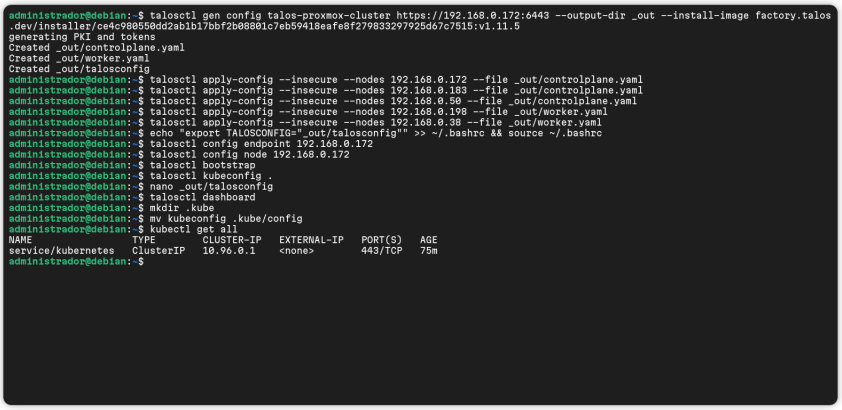


Figura 4.56: Arranque del clúster, obtención de *kubeconfig*, configuración de *talosctl*



Figura 4.57: Modificación realizada en _out/talosconfig

Si ejecutamos *talosctl dashboard*, podemos ver información sobre los nodos, de la misma forma que se muestra por pantalla. Con esto ya tenemos el clúster de Kubernetes levantado.

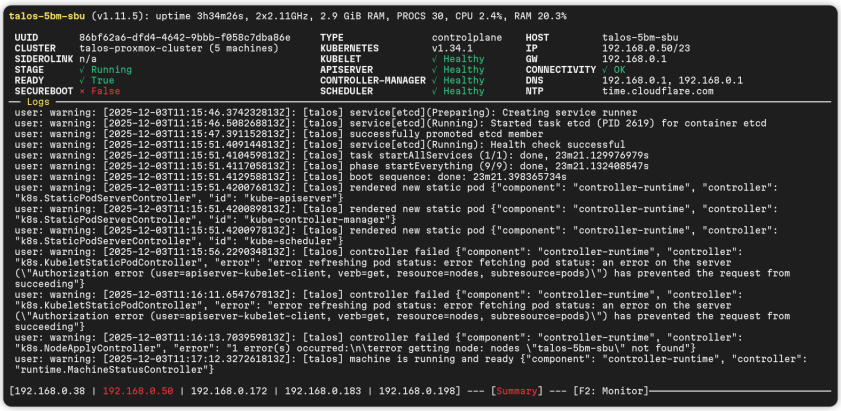


Figura 4.58: Resultado de ejecutar *talosctl dashboard*

4.6 Configuración del almacenamiento del clúster

Por defecto, el almacenamiento de las imágenes de contenedores es efímero y no persistente, por lo que se queda dentro de la imagen, sólo pueden ser accedido por ellas y cuando desaparecen, se eliminan los datos. Como KubeFlow se despliega sobre Kubernetes y está formado por contenedores, éstos necesitan una forma de almacenar datos de forma permanente. Esto se realiza de forma dinámica con *PersistentVolumeClaims* (*Kubernetes Persistent Volume Claims* (2025)), que serían solicitudes de *PersistentVolumes* (*Kubernetes Persistent Volumes* (2025)), lugares donde se puede almacenar información de forma permanente en Kubernetes.

Para que el clúster tenga persistencia de datos, debemos de configurar una *StorageClass*

(*Kubernetes Storage Classes* (2025)), que permitirá a Kubernetes tener un lugar donde alojar estos volúmenes persistentes. Para ello, utilizaremos un almacenamiento conectado a la red, con NFS. En este trabajo se utilizará el que hemos creado dentro de nuestro TrueNAS virtualizado, pero en el entorno real del laboratorio podría usarse una ruta de red ya existente o un NAS dedicado para el servidor.

Para que nuestro clúster de Kubernetes pueda conectarse a dicho almacenamiento de red, debemos instalar un *driver* que nos permita conectarnos, indicarle dónde se encuentra carpeta de red, cómo acceder a y que debe hacer con ella.

Lo primero es añadir a nuestro clúster de Kubernetes el *driver* de NFS CSI. Para ello, añadimos el repositorio de helm donde se encuentra el driver, y después lo instalamos.

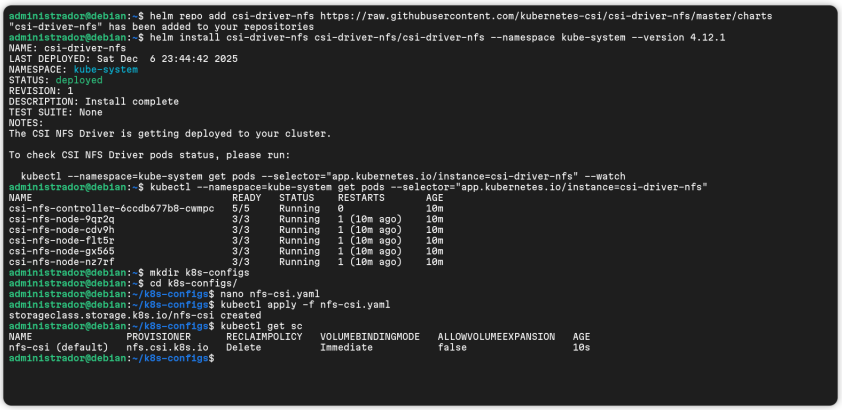


Figura 4.59: Instalación y arranque del *driver* NFS CSI

Para ello, utilizaremos el fichero *nfs-csi.yaml*, cuyos contenidos se pueden ver en el código 4.1.

Código 4.1: Contenido de *nfs-csi.yaml*

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-csi
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: nfs.csi.k8s.io
parameters:
  server: 192.168.0.201
  share: /mnt/pool1/kubernetes
reclaimPolicy: Delete
volumeBindingMode: Immediate
mountOptions:
  - nolock
```

Antes de continuar, podemos verificar que nuestro almacenamiento funciona correctamente creando un *PresistentVolume* y una *PersistentVolumeClaim*. Primero crearemos el *Pre-*

sistentVolume, que tenemos definido en *foo-pv.yaml*, después el *PersistentVolumeClaim* en *foo-pvc.yaml*, y comprobaremos el estado en que se encuentran. Una vez comprobado que el *StorageClass* funciona correctamente (Tanto el *PersistentVolume* como la *PersistentVolume-Claims* aparecen *Bound*), los eliminamos.

Código 4.2: Contenido de *foo-pv.yaml*

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: foo-pv
spec:
  capacity:
  storage: 25Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: nfs-csi
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /mnt/pool1/kubernetes
    server: 192.168.0.201
```

Código 4.3: Contenido de *foo-pvc.yaml*

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: foo-pvc
spec:
  volumeName: foo-pv
  accessModes:
    - ReadWriteMany
resources:
  requests:
    storage: 10Gi
```

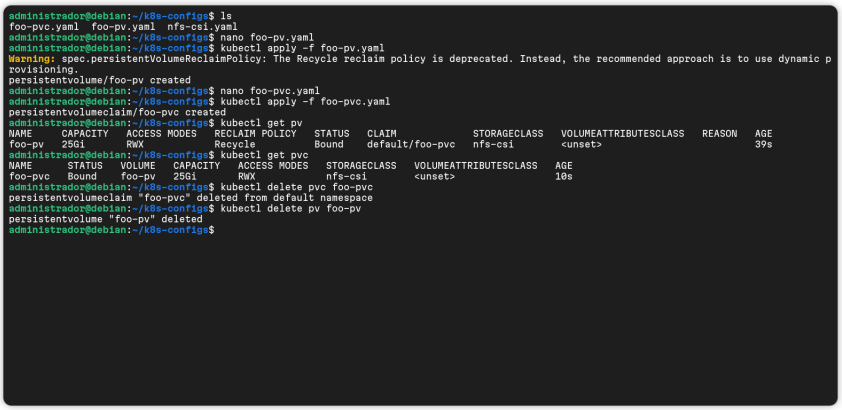


Figura 4.60: Comprobación del funcionamiento de la *StorageClass*

Con esto ya tendríamos el almacenamiento del clúster configurado.

4.7 Arranque de *KubeFlow*

Para arrancar *KubeFlow*, debemos de copiar el repositorio oficial donde se encuentran los *manifests* de despliegue (*Kubeflow Manifests* (2025)), entrar en la carpeta, ubicarnos en la última verión (en nuestro caso la v1.11) con *git* y desplegar el sistema con *while ! kustomize build example | kubectl apply –server-side –force-conflicts -f -; do echo "Retrying to apply resources"; sleep 20; done*. Tardará varios minutos en desplegarse (en mi caso llegó a unos 10 minutos), debemos dejar el comando ejecutándose. Una vez termine, ya tendremos el sistema despelgado.

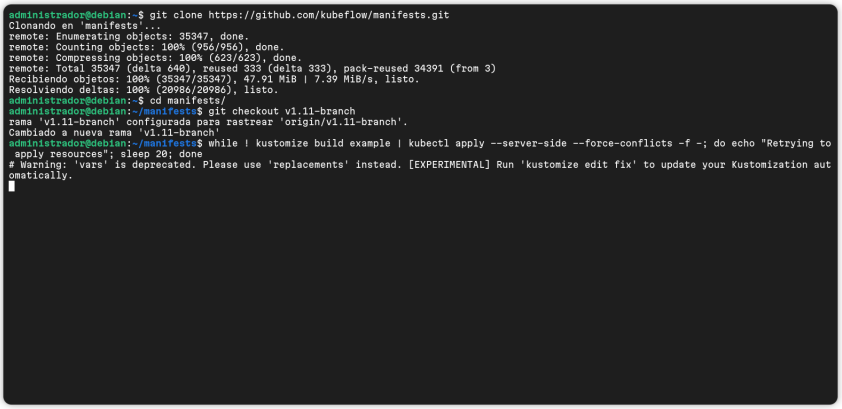


Figura 4.61: Contenido de *Despliegue de KubeFlow*

Para comprobar que el sistema se ha desplegado completamente, haremos un reenvío de puertos del *frontend* del sistema, con *kubectl port-forward svc/istio-ingressgateway -n istio-*

system 8080:80.

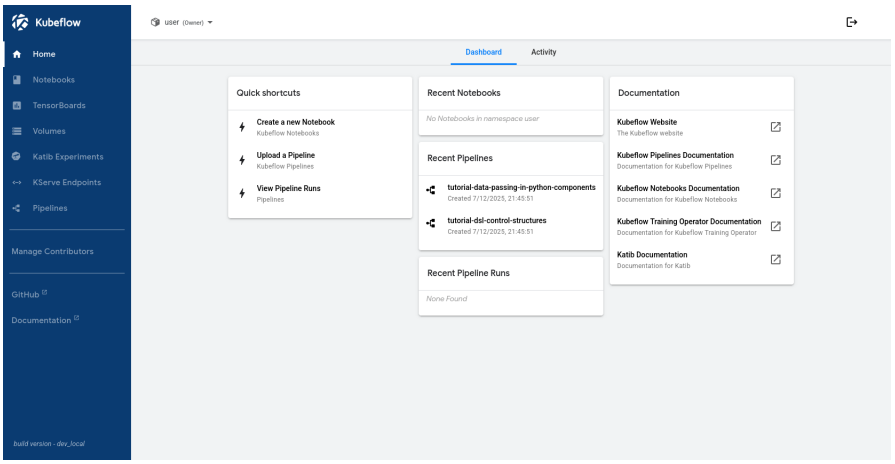


Figura 4.62: Pantalla de bienvenida de Kubeflow

4.8 Gestión de usuarios

Para la gestión de los usuarios, se deberá conectar Kubeflow a nuestro proveedor de autentificación a través de Dex. El proceso se encuentra documentado en la documentación oficial (*Dex | Documentation* (2025)).

5 Conclusiones

A lo largo de este Trabajo de Fin de Grado se ha realizado el estudio, diseño y despliegue de un entorno orientado a la ejecución de modelos de *Deep Learning* de manera colaborativa en una máquina local, junto con una implementación parcial en el laboratorio de la universidad.

Durante el desarrollo se investigaron las diferentes tecnologías existentes relacionadas con la virtualización, gestión y orquestración de contenedores, así como diversas distribuciones de Linux. Por último se comentan dos de los sistemas más populares relacionados con MLOps.

Finalmente, se optó por una arquitectura basada en Proxmox VE, Talos Linux y Kubeflow, que demostró ofrecer la robustez, reproducibilidad y capacidad de administración necesarias. Con esta arquitectura, en la solución final se ha conseguido alcanzar el objetivo principal de implementar un sistema para el entrenamiento de modelos de Inteligencia Artifical de forma colaborativa.

Respecto a los subobjetivos específicos, se ha logrado implementar el sistema de forma robusta y escalable. Se encuentra documentado, y proporciona gestión de usuarios con capacidad de integración con el sistema de usuarios qu see utilize el departamento, pero no se ha realizado una demostración.

El resultado final del proyecto es una guía completa de instalación y despliegue del entorno Kubeflowm documentada paso a paso. Como trabajo futuro se querría desplegar completamente en el sistema final, que es el equipo del Departamento de Tecnología Informática y Computación de la Universidad de Alicante.

Bibliografía

About oracle virtualbox. (2025). Descargado de https://web.archive.org/web/20250831110251/https://www.virtualbox.org/manual/topics/Introduction.html (Último acceso: 2025-12-09)

Azure kubernetes service (aks). (2025). Descargado de https://web.archive.org/web/20251209210432/https://azure.microsoft.com/en-us/products/kubernetes-service/ (Último acceso: 2025-12-09)

The debian gnu/linux faq. (2024). Descargado de https://web.archive.org/web/20250531111241/https://www.debian.org/doc/manuals/debian-faq/debian-faq.en.pdf (Último acceso: 2025-12-09)

Dex | documentation. (2025). Descargado de https://web.archive.org/web/20251210160317/https://dexidp.io/docs/ (Último acceso: 2025-12-10)

The effortless kubernetes. (2025). Descargado de https://web.archive.org/web/20251209205244/https://canonical.com/microk8s (Último acceso: 2025-12-09)

Fedora coreos frequently asked questions. (2025). Descargado de https://web.archive.org/web/20251209195053/https://docs.fedoraproject.org/en-US/fedora-coreos/faq/ (Último acceso: 2025-12-09)

Flatcar container linux. (2025). Descargado de https://web.archive.org/web/20251209195010/https://www.flatcar.org/ (Último acceso: 2025-12-09)

Google kubernetes engine (gke). (2025). Descargado de https://web.archive.org/web/20251209210434/https://cloud.google.com/kubernetes-engine (Último acceso: 2025-12-09)

Hyper-v server 2019. (2025). Descargado de https://web.archive.org/web/20251205162646/https://learn.microsoft.com/en-us/lifecycle/products/hyperv-server-2019 (Último acceso: 2025-12-05)

Hyper-v virtualization in windows server and windows. (2025). Descargado de https://web.archive.org/web/20251205162716/https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/overview (Último acceso: 2025-12-05)

An industry-standard container runtime with an emphasis on simplicity, robustness and portability. (2025). Descargado de https://web.archive.org/web/20251209193711/https://containerd.io/ (Último acceso: 2025-12-09)

Introduction | rke2. (2025). Descargado de https://web.archive.org/web/20251209205924/https://docs.rke2.io/ (Último acceso: 2025-12-09)

Iommu architectural specification. (2011). Descargado de https://web.archive.org/web/20140110200924/http://developer.amd.com/wordpress/media/2012/10/48882.pdf (Último acceso: 2025-12-07)

K3s. (2025). Descargado de https://web.archive.org/web/20251209204941/https://k3s.io/ (Último acceso: 2025-12-09)

kind. (2025). Descargado de https://web.archive.org/web/20251209204623/https://kind.sigs.k8s.io/ (Último acceso: 2025-12-09)

Kreuzberger, D., Kühl, N., y Hirschl, S. (2023). Machine learning operations (mlops): Overview, definition, and architecture. *IEEE Access*, *11*, 31866-31879. doi: 10.1109/ACCESS.2023.3262138

Kubeflow manifests. (2025). Descargado de https://github.com/kubeflow/manifests (Último acceso: 2025-12-06)

Kubernetes components. (2024). Descargado de https://web.archive.org/web/20250510043530/https://kubernetes.io/docs/concepts/overview/components/ (Último acceso: 2025-12-05)

Kubernetes overview. (2024). Descargado de https://web.archive.org/web/20250511133909/https://kubernetes.io/docs/concepts/overview/ (Último acceso: 2025-12-05)

Kubernetes persistent volume claims. (2025). Descargado de https://web.archive.org/web/20251210184623/https://kubernetes.io/docs/concepts/storage/persistent-volumes/#persistentvolumeclaims (Último acceso: 2025-12-10)

Kubernetes persistent volumes. (2025). Descargado de https://web.archive.org/web/20251210184623/https://kubernetes.io/docs/concepts/storage/persistent-volumes/ (Último acceso: 2025-12-10)

Kubernetes storage classes. (2025). Descargado de https://web.archive.org/web/20251210184618/https://kubernetes.io/docs/concepts/storage/storage-classes/ (Último acceso: 2025-12-10)

Laadan, O., y Nieh, J. (2010, 05). Operating system virtualization: Practice and experience.. doi: 10.1145/1815695.1815717

Lightweight container runtime for kubernetes. (2023). Descargado de https://web.archive.org/web/20251209210558/https://cri-o.io/ (Último acceso: 2025-12-09)

minikube | welcome! (2025). Descargado de https://web.archive.org/web/20251209204703/https://minikube.sigs.k8s.io/docs/ (Último acceso: 2025-12-09)

Proxmox ve | installation. (2025). Descargado de https://web.archive.org/web/20251210171254/https://pve.proxmox.com/wiki/Installation (Último acceso: 2025-12-10)

Proxmox virtual environment. (2025). Descargado de https://web.archive.org/web/20251205170008/https://www.proxmox.com/en/products/proxmox-virtual-environment/overview (Último acceso: 2025-12-05)

Rancher. (2025). Descargado de https://web.archive.org/web/20251209210123/https://www.rancher.com/ (Último acceso: 2025-12-09)

Red hat enterprise linux. (2025). Descargado de https://web.archive.org/web/20251209195231/https://www.redhat.com/en/technologies/linux-platforms/enterprise-linux (Último acceso: 2025-12-09)

Red hat openshift. (2025). Descargado de https://web.archive.org/web/20251209210126/https://www.redhat.com/en/technologies/cloud-computing/openshift (Último acceso: 2025-12-09)

Red hat openshift kubernetes engine. (2025). Descargado de https://web.archive.org/web/20251209205531/https://www.redhat.com/en/technologies/cloud-computing/openshift/kubernetes-engine (Último acceso: 2025-12-09)

Rodríguez-Haro, F., Freitag, F., Navarro, L., Hernánchez-sánchez, E., Farías-Mendoza, N., Guerrero-Ibáñez, J. A., y González-Potes, A. (2012). A summary of virtualization techniques. *Procedia Technology*, *3*, 267-272. Descargado de https://www.sciencedirect.com/science/article/pii/S2212017312002587 (The 2012 Iberoamerican Conference on Electronics Engineering and Computer Science) doi: https://doi.org/10.1016/j.protcy.2012.03.029

Suse linux enterprise server. (2025). Descargado de https://web.archive.org/web/20251209203933/https://www.suse.com/products/server/ (Último acceso: 2025-12-09)

Ubuntu server documentation. (2025). Descargado de https://web.archive.org/web/20251209203957/https://documentation.ubuntu.com/server/ (Último acceso: 2025-12-09)

Vmware esxi. (2024). Descargado de https://web.archive.org/web/20240720073201/https://www.vmware.com/products/esxi-and-esx.html (Último acceso: 2025-12-07)

Vmware fusion & workstation. (2025). Descargado de https://web.archive.org/web/20251011181254/https://www.vmware.com/docs/desktop-hypervisor-faqs (Último acceso: 2025-12-09)

What is a hypervisor? (2023). Descargado de https://web.archive.org/web/20251205165711/https://www.redhat.com/en/topics/virtualization/what-is-a-hypervisor (Último acceso: 2025-12-05)

What is amazon eks? (2025). Descargado de https://web.archive.org/web/20250624083537/https://docs.aws.amazon.com/en_us/eks/latest/userguide/what-is-eks.html (Último acceso: 2025-12-09)

What is kubeflow? (2025). Descargado de https://web.archive.org/web/20251209160638/https://cloud.google.com/discover/what-is-kubeflow (Último acceso: 2025-12-09)

What is mlflow? (2024). Descargado de https://web.archive.org/web/20251209160433/https://mlflow.org/docs/latest/ml/ (Último acceso: 2025-12-09)

What is talos linux? (2025). Descargado de https://web.archive.org/web/20251209194801/https://docs.siderolabs.com/talos/v1.11/overview/what-is-talos (Último acceso: 2025-12-09)

Zero friction kubernetes. (2025). Descargado de https://web.archive.org/web/20251210030307/https://k0sproject.io/ (Último acceso: 2025-12-10)

Lista de Acrónimos y Abreviaturas

AMD64	*Arquitectura x86-64, x64 o AMD64.*
API	*Application Programming Interface.*
CNCF	*Cloud Native Computing Foundation.*
CRI	*Container Runtime Interface.*
GNU AGPLv3	*GNU Affero General Public License, version 3.*
IOMMU	*Input Output Memory Management Unit.*
K8s	Kubernetes.
KVM	*Kernel-based Virtual Machine.*
LDAP	*Lightweight Directory Access Protocol.*
LXC	*Linux Containers.*
ML	*Machine Learning.*
MLOps	*Machine Learning Operations.*
OAuth 2.0	*Open Authorization 2.0.*
Proxmox VE	*Proxmox Virtual Environment.*
SAML 2.0	*Security Assertion Markup Language 2.0.*
XMP	*Extreme Memory Profile.*

z